Grundlagen der künstlichen Intelligenz

Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas

Universidad de Granada

Grundlagen der künstlichen Intelligenz

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2026

Índice general

1. Introduction

Artificial Intelligence (AI) is a field that is rapidly evolving and has the potential to revolutionize various industries. Before delving into the technical aspects of AI, it is important to understand the concept of intelligence and how it can be measured.

1.1. Intelligence Tests

Scientists have tried to decide what differentiates AI from computer algorithms; i.e. what is the essence of intelligence? This question has been debated for decades, and there are different tests that have been proposed to determine whether a machine can be considered intelligent or not. The importance of these tests is shown in the Chinese Room thought experiment.

- Chinese Room Experiment:

  The Chinese Room thought experiment involves a person who does not understand Chinese sitting in a room with a set of instructions for how to respond to Chinese characters. The person receives Chinese characters as input and uses the instructions to produce appropriate responses, which are then sent back out of the room.

  From the outside, it may appear that the person in the room understands Chinese, but in reality, they are simply following a set of rules without any comprehension of the language. This thought experiment raises questions about whether a machine can truly understand language or if it is simply following a set of instructions.

Some of the most well-known tests include:

- Turing Test:

  In the Turing Test, a human evaluator interacts with both a machine and a human without knowing which is which. If the evaluator cannot reliably distinguish between the machine and the human, then the machine is said to have passed the Turing Test and is considered to be intelligent.

  It only tests for the ability to mimic human behavior, not for true understanding or consciousness.

- Winograd Test:

  The test involves a series of sentences known as "Winograd Schemas". They include ambiguous pronouns that require common sense and contextual understanding to resolve.

This test is not about factual knowledge but rather about understanding context.

- Lovelace Test:

  The Lovelace Test is designed to evaluate a machine's ability to create something original and surprising.

  This test is extremely difficult for machines to pass, as they often just combine existing ideas rather than creating something truly new.

- Metzinger Test:

  The Metzinger Test is designed to evaluate a machine's ability to have a subjective experience, a sense of "self, I", and consciousness. There is no definitive method for conducting the test in practice, since subjective experience cannot be observed from the outside.

1.2. Initial Definitions

From more general concepts to more specific ones, where each one builds upon the previous, some important terms in the field of AI include:

- Data Science: Field that turns data into knowledge and decisions.
- Artificial Intelligence: Field that aims to create machines that can perform tasks that typically require human intelligence.
- Machine Learning: Subfield of AI that focuses on training machines to learn from data and improve their performance over time without being explicitly programmed.
- Neural Networks: A type of (usually supervised) machine learning model inspired by the structure and function of the human brain. They consist of layers of interconnected nodes (neurons) that process and transmit information.
- Deep Learning: Neural networks with multiple layers. Deep learning has been particularly successful in tasks such as image recognition, natural language processing, and speech recognition.

2. Data Science

Data Science is a multidisciplinary field that combines statistical analysis, machine learning, and domain expertise to extract insights and knowledge from data.

2.1. Statistics

Data science involves, in a large scale, a lot of statistics, which is the branch of mathematics that deals with the collection, analysis, interpretation, presentation, and organization of data. Some of the most important concepts in statistics include:

- Outlier: A data point that is significantly different from the other data points in the dataset. Outliers can be caused by errors in data collection or by natural variability in the data. They can have a significant impact on the results of statistical analysis, so it is important to identify and handle them appropriately.

- Mean: The average of a set of numbers. It represents the central tendency of the data.

$$\text{Mean} = E[X] = \mu := \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Median: The middle value of a set of numbers when they are arranged in order. It is less affected by outliers than the mean. Other terms as percentiles and quartiles are also used to describe the distribution of the data. The p-th percentile is the value below which $p\,\%$ of the data falls. The first quartile (Q1) is the 25th percentile, the second quartile (Q2) is the 50th percentile (which is also the median), and the third quartile (Q3) is the 75th percentile.

- Mode: The value that appears most frequently in a dataset. It is useful for categorical data.

- Variance: A measure of how much the data varies from the mean. It is calculated as the average of the squared differences from the mean.

$$\text{Variance} = E[(X-\mu)^2] = \sigma^2 := \frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2$$

  It can be however calculated using the Besel correction, which divides by $n-1$ instead of n to provide an unbiased estimator of the population variance when working with a sample. This is because when we calculate the sample mean,

we are using the same data points that we are trying to estimate the variance for, which can lead to an underestimation of the true population variance. By dividing by $n-1$, we are effectively increasing the variance estimate to account for this bias.

$$\text{Sample Variance} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \mu)^2$$

- Standard Deviation: The square root of the variance. It is a measure of how much the data varies from the mean, but it is in the same units as the data.

$$\text{Standard Deviation} = \sqrt{\text{Variance}} = \sigma$$

- Skewness: A measure of the asymmetry of the distribution of the data. A positive skew indicates that the data is skewed to the right, which means that the mean is greater than the median, meaning that there are more values on the right side of the distribution than on the left. A negative skew indicates that the data is skewed to the left, which means that the mean is less than the median, meaning that there are more values on the left side of the distribution than on the right. A skewness of zero indicates that the data is perfectly symmetrical.

- Kurtosis: A measure of the "tailedness.of the distribution of the data. A high kurtosis indicates that the data has heavy tails, which means that there are more extreme values in the data than would be expected in a normal distribution. A low kurtosis indicates that the data has light tails, which means that there are fewer extreme values in the data than would be expected in a normal distribution. A kurtosis of zero indicates that the data has the same tailedness as a normal distribution.

- Covariance: A measure of how much two random variables change together. It is calculated as:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

  A positive covariance indicates that the two variables tend to increase or decrease together, while a negative covariance indicates that one variable tends to increase when the other decreases. A covariance of zero indicates that there is no linear relationship between the two variables.

- Correlation: A measure of the strength and direction of the linear relationship between two random variables. It is calculated as:

$$\text{Correlation}(X, Y) = r_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

  where σ_X and σ_Y are the standard deviations of X and Y, respectively. The correlation coefficient ranges from -1 to 1, where a value of 1 indicates a perfect positive linear relationship, a value of -1 indicates a perfect negative linear relationship, and a value of 0 indicates no linear relationship between the two variables.

It should be noted that correlation does not imply causation. Just because two variables are correlated does not mean that one variable causes the other to change. There may be other factors at play that are influencing both variables, or the correlation may be purely coincidental.

2.2. Data

Data is the raw material that fuels the development of AI systems, and understanding how to work with data is crucial for anyone interested in the field of AI. Two main people work on it:

- Data Engineer: Responsible for building and maintaining the infrastructure that allows for the collection, storage, and processing of data.
- Data Scientist: Responsible for analyzing and interpreting data to extract insights and knowledge.

When working with data, the PPDAC Data Cycle is often used as a framework to guide the process. The PPDAC Data Cycle consists of five stages:

- Problem: Defining the problem that needs to be solved, stating the objectives and questions that need to be answered.
- Plan: Developing a plan for how to answer the questions and achieve the objectives. This includes deciding on which data to collect, how to collect it, and how to analyze it.
- Data: Collecting the data according to the plan. This can involve gathering data from various sources, such as databases, APIs, or web scraping. It also involves preparing the data for analysis, which may include cleaning, transforming, and encoding the data.
- Analysis: Analyzing the data to extract insights and knowledge. This can involve using various statistical and machine learning techniques to identify patterns and relationships in the data.
- Conclusion: Drawing conclusions from the analysis and communicating the results. This can involve creating visualizations, writing reports, or presenting the findings to stakeholders.

2.2.1. Data Collection

Regarding data collection, there are some aspects that must be taken into account:

- Data Quality: The quality of the data is crucial for the success of any AI project.
- Data Integrity: Ensuring that the data is accurate and reliable.
- Data Consistency: Ensuring that the data is consistent across different sources and formats.

2.2.2. Data Preprocessing

Once the data has been collected, it must be preprocessed before it can be used for training AI models. This involves:

- Data Validation: Checking the data for errors and inconsistencies. Depending on the source and type of data, some rules can be applied to validate it.
- Data Sorting: Organizing the data in a way that makes it easier to work with. This can involve sorting the data by a specific column.
- Data Aggregation: Combining data from multiple sources or records into a single record. This can involve calculating summary statistics, such as the mean or median, for a group of records. This is often used to reduce the size of the data and to make it easier to analyze. However, it can also lead to loss of information, so it should be used with caution.
- Data Cleaning: Removing or correcting any errors or inconsistencies in the data.
- Data Transformation: Converting the data into a format that can be used for training AI models.

Data preprocessing also involves data scaling, which is the process of normalizing or standardizing the data to ensure that it is on the same scale. This is important because many AI algorithms are sensitive to the scale of the data and can perform poorly if the data is not scaled properly. Some common scalers include:

- `Binarizer`: Converts categorical data into binary data. A threshold is set, and values above the threshold are converted to 1, while values below the threshold are converted to 0.

$$X' = \begin{cases} 0 & \text{if } X \leqslant \text{threshold} \\ 1 & \text{if } X > \text{threshold} \end{cases}$$

- `MaxAbsScaler`: Scales the data to the range [-1, 1] by dividing each value by the maximum absolute value in the data.

$$X' = \frac{X}{\text{máx}(|X|)}$$

- `MinMaxScaler`: Scales the data to a specified range, let's say [mín, máx]. Firsrly the data is scaled to the range [0, 1] (X_{std}), and then it is scaled to the specified range.

$$X_{\text{std}} = \frac{X - \text{mín}(X)}{\text{máx}(X) - \text{mín}(X)} \qquad X' = X_{\text{std}} \cdot (\text{máx} - \text{mín}) + \text{mín}$$

- `Normalizer`: Scales the data to have a unit norm. This is done by dividing each value by the norm of the data. Usually, the L2 norm is used.

$$X' = \frac{X}{\|X\|}$$

- `PowerTransformer`: Applies a power transformation to the data to make it more Gaussian-like. This is useful for data that is not normally distributed.
- `RobustScaler`: Scales the data using statistics that are robust to outliers. This is useful for data that contains outliers.

$$X' = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

  where $\text{IQR}(X)$ is the interquartile range of X.
- `StandardScaler`: Scales the data to have a mean of 0 and a standard deviation of 1.

$$X' = \frac{X - \mu}{\sigma}$$

  where μ is the mean of X and σ is the standard deviation of X.

One important aspect of data preprocessing is *encoding* the data. This is the process of converting categorical data into numerical data that can be used for training AI models. There are several techniques for encoding data, such as:

- `Ordinal Encoding`: Assigning a unique integer to each category in the data. The order of the integers is important, as it implies a relationship between the categories.

$$\{\text{small}, \text{medium}, \text{large}\} \to \{0, 1, 2\}$$

- `Label Encoding`: Similar to ordinal encoding, but it does not imply any relationship between the categories. Each category is assigned a unique integer, but the order of the integers does not matter.

$$\{\text{red}, \text{green}, \text{blue}\} \to \{0, 1, 2\}$$

- `One-Hot Encoding`: Creating a binary column for each category in the data.

$$\{\text{red}, \text{green}, \text{blue}\} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- `Dummy Encoding (Leave out the last)`: Similar to one-hot encoding, but it leaves out the last category to avoid multicollinearity.

$$\{\text{red}, \text{green}, \text{blue}\} \to \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

- `Absolute Frequency Encoding`: Assigning a unique integer to each category based on the absolute frequency of the category in the data. Once drawback of this technique is that, if two categories have the same frequency, they will be assigned the same integer, which can lead to confusion.

$$\{\text{red}, \text{red}, \text{red}, \text{green}, \text{blue}, \text{blue}\}$$
$$\{\text{red}, \text{green}, \text{blue}\} \to \{3, 1, 2\}$$

- `Relative Frequency Encoding`: Assigning a unique integer to each category based on the relative frequency of the category in the data.

$$\{\text{red}, \text{red}, \text{red}, \text{green}, \text{blue}, \text{blue}\}$$
$$\{\text{red}, \text{green}, \text{blue}\} \rightarrow \{3/6, 1/6, 2/6\}$$

- `Feature Hash Encoding`: Hashing the categories into a fixed number of columns. This is useful for data with a large number of categories.

2.2.3. Data Usage

Once the data has been preprocessed, they can be used. There are two main destinations for the data:

- Data Visualization: The process of creating visual representations of data to help understand and communicate insights. This is used by people.

  The Gestalt Principles of Perception are a set of principles that describe how people perceive and organize visual information, and they can be applied to data visualization to create more effective and engaging visualizations.

- Data Modeling: The process of using data to train AI models. This is used by machines.

3. Search Algorithms

In our current world, plenty of algorithms are used to search for information. From the most basic ones, such as sorting algorithms, to the most complex ones, such as search engines. In this chapter, we will explore some of the most common search algorithms and their applications.

3.1. Graph Theory

Search algorithms are often used in graphs, as they provide a way to represent complex relationships between data. A graph $G = (V, E)$ is defined as a pair of sets, where V is called the set of vertices and E is called the set of edges. The vertices represent the data points, while the edges represent the relationships between them. There are different types of graphs, such as directed and undirected graphs, weighted and unweighted graphs, or cyclic and acyclic graphs. Each type of graph has its own properties and algorithms that can be applied to it. An example graph is shown in Figure 3.1.

A graph is usually represented as an *adjacency matrix*. Where the rows represent the origin vertices, the columns represent the destination vertices[1], and the values a_{ij} represent:

- If the graph is unweighted, a_{ij} represents the number of edges from vertex i to vertex j.
- If the graph is weighted, a_{ij} represents the weight of the edge from vertex i to vertex j. If there is no edge, a_{ij} is usually set to infinity or an invalid value.

If there are a lot of vertices that are not connected to each other, the adjacency matrix can be very sparse, meaning that it contains a lot of zeros. In this case, it is

[1]This could also be the opposite: the origin vertices could be the colums and the destination vertices, the rows.

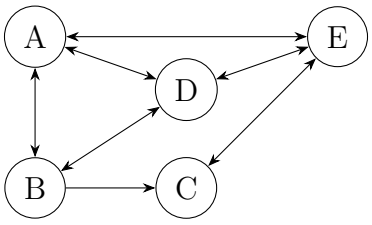

Figura 3.1: Example of a directed graph.

```
adjacency_list = {
    'A': ['B', 'D', 'E'],
    'B': ['A', 'C', 'D'],
    'C': ['B', 'E'],
    'D': ['A', 'B', 'E'],
    'E': ['A', 'C', 'D']
}
```

Código fuente 1: Adjacency list for the graph in Figure 3.1.

more efficient to use an *adjacency list* to represent the graph, where each vertex is represented as a list of its neighboring vertices. This can save a lot of space, especially for large graphs with few edges. For example, the adjacency matrix for the graph in Figure 3.1 is shown in Equation 3.1, while the adjacency list is Listing 1.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix} \tag{3.1}$$

This matrix is also called "next-neighbor" matrix, as the neighboring vertices of a vertex i can be found by looking at the i-th row of the matrix. The "next-next-neighbor" matrix, on the other hand, is obtained by multiplying the adjacency matrix by itself. This matrix can be used to find the vertices that are two steps away from a given vertex.

Lastly, the *transition matrix* is obtained by normalizing the adjacency matrix. This is done by dividing each entry a_{ij} by the sum of the entries in the i-th row. The transition matrix can be used to find the probability of transitioning from one vertex to another in a random decision process.

3.1.1. PageRank

PageRank is an algorithm used to determine the importance of nodes in a graph. It is well-known for being used by Google Search to rank web pages in their search engine results. It works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites. Everything is represented as a directed graph, where the vertices represent web pages and the edges represent hyperlinks between them. There are certain parameters that can be adjusted in the PageRank algorithm, such as:

- The teleportation probability vector. It's n-th entry represents the probability of jumping to the n-th page when a random surfer decides to teleport (going to a random page instead of following a link). It is usually set to a uniform distribution, meaning that the random surfer has an equal probability of jumping to any page.

- The damping factor α, which represents the probability that a random surfer will continue following links rather than teleporting.
- The convergence threshold, which determines when the algorithm should stop iterating and return the PageRank values.

The algorithm returns the PageRank vector, whose n-th entry represents the PageRank of the n-th page. The PageRank values decide in which order the pages are shown in the search results, with higher PageRank values indicating more important pages. The PageRank algorithm works as follows:

1. Initialize the PageRank vector with the teleportation probability vector.
2. Update the old PageRank vector (PR) by adding the transition matrix multiplied by the old PageRank vector (weighted by the damping factor α) and the teleportation probability vector (weighted by $1 - \alpha$):

   $$PR = \alpha \cdot M \cdot PR + (1 - \alpha) \cdot v$$

   Where M is the transition matrix and v is the teleportation probability vector.
3. Repeat the process until the change in the PageRank vector is less than the convergence threshold (or a maximum number of iterations is reached).

Once the algorithm converges, the PageRank values can be used to rank the web pages in search results. The higher the PageRank value, the more important the page is considered to be, and therefore, it will be ranked higher in the search results.

3.2. Sorting Algorithms

Sorting algorithms are a fundamental part of computer science, as they allow us to organize data in a specific order. In addition, they are used in many search algorithms, as they can help to reduce the search space and improve the efficiency of the search.

3.2.1. Insertion Sort

This algorithm works by dividing the array into a sorted and an unsorted part. The sorted part is initially empty, and the unsorted part contains all the elements.

1. Start with the first element of the unsorted part and compare it with the last element of the sorted part.
2. IMove all the elements of the sorted part that are greater than the current element to the right.
3. Insert the current element into its correct position in the sorted part.
4. Repeat the process for all the elements in the unsorted part until it is empty.

It has a time complexity of $O(n^2)$.

3.2.2. Bubble Sort

This algorithm works by repeatedly swapping adjacent elements if they are in the wrong order. The process is repeated until the array is sorted.

1. Compare each pair of adjacent elements in the array.
2. If the elements are in the wrong order, swap them.
3. Repeat the process for all the elements in the array until it is sorted.

It has a time complexity of $O(n^2)$.

3.2.3. Shaker Sort

This algorithm is a variation of Bubble Sort that sorts in both directions on each pass through the list. It works by comparing adjacent elements and swapping them if they are in the wrong order, similar to Bubble Sort. However, it also compares elements in the opposite direction, which can help to reduce the number of passes needed to sort the array.

1. Start at the beginning of the array and compare each pair of adjacent elements.
2. If the elements are in the wrong order, swap them.
3. After reaching the end of the array, reverse the direction and compare each pair of adjacent elements again.
4. Repeat the process until the array is sorted.

It has a time complexity of $O(n^2)$, but with lower constant factors than Bubble Sort.

3.2.4. Quick Sort

This divide-and-conquer algorithm works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. This process continues until the base case of an empty array or an array with a single element is reached.

1. Choose a pivot element from the array.
2. Partition the array into two sub-arrays: one with elements less than the pivot and one with elements greater than the pivot.
3. At this point, the pivot is in its final position.
4. Recursively apply the above steps to the sub-arrays until the base case is reached.

The efficiency of Quick Sort depends on the choice of the pivot. The complexity depends on the partitioning of the array. If the pivot divides the array into two equal halves, the time complexity is $O(n \log n)$. However, if the pivot is consistently the smallest or largest element, the time complexity degrades to $O(n^2)$.

3.2.5. Topological Sorting: Kahn's Algorithm

This algorithm is used to sort the vertices of a directed acyclic graph (DAG) in a linear order. If there is a directed edge from vertex u to vertex v, then u comes before v in the ordering. It is used in several applications, such as scheduling tasks, resolving dependencies, and determining the order of compilation in programming languages. The algorithm works as follows:

1. The result is going to be stored in a list called L.
2. Find all vertices with no incoming edges and add them to a set S.
3. While S is not empty:
   a) Remove a vertex n from S and add it to L.
   b) For each vertex m with an edge from n to m:
      1) Remove the edge from n to m.
      2) If m has no other incoming edges, add m to S.
4. If there are still edges in the graph, it means that there is a cycle, and therefore, a topological sort is not possible. Otherwise, the list L contains the vertices in topologically sorted order.

3.3. Uninformed Search

Uninformed search algorithms are those that do not have any additional information about the search space. They explore the search space in a systematic way, without any guidance or heuristics.

3.3.1. Breadth-First Search

This algorithm explores the search space level by level, starting from the root node and expanding all the nodes at the present depth before moving on to the nodes at the next depth level. It uses a queue (FIFO) to keep track of the nodes to be explored. It is guaranteed to find the shortest path, but it can be memory intensive, as it needs to store all the nodes at the current depth level.

3.3.2. Depth-First Search

This algorithm explores the search space by going as deep as possible along each branch before backtracking. It uses a stack (LIFO) to keep track of the nodes to be explored. It is not guaranteed to find the shortest path, but it can be more memory efficient than breadth-first search.

3.3.3. Depth-Limited Search

This algorithm is a variation of depth-first search that limits the depth of the search. It is used to avoid infinite loops in cases where the search space is infinite or has cycles. It uses a stack (LIFO) to keep track of the nodes to be explored, and it keeps track of the current depth of the search. If the depth limit is reached, it backtracks and explores other branches. It is not guaranteed to find the searched node, but it can be more memory efficient than depth-first search. The choice of the depth limit is crucial, as it can affect the completeness and optimality of the search.

3.3.4. Iterative Deepening Search

This algorithm combines the benefits of both breadth-first and depth-first search. It performs a depth-limited search with increasing depth limits until a solution is found.

1. Start with a depth limit of 0.
2. Perform a depth-limited search with the current depth limit.
3. If a solution is found, return it. Otherwise, increase the depth limit and repeat the process.

A stack (LIFO) is used to keep track of the nodes to be explored in the depth-limited search. It is guaranteed to find the shortest path, and it can be more memory efficient than breadth-first search.

3.3.5. Bidirectional Search

This algorithm is used to find the shortest path between two nodes in a graph. It works by simultaneously searching (typically BFS) from both the start node and the goal node until the two searches meet in the middle. It uses two data structures to keep track of the nodes to be explored from both directions. It is guaranteed to find the shortest path, and it can be more efficient than unidirectional search, especially in cases where the search space is large.

3.3.6. Uniform Cost Search

This algorithm is used in weighted graphs (where the edges have positive costs) to find the least costly path from the root node to a goal node. It is similar to breadth-first search, but it takes into account the cost of the path. It uses a priority queue to keep track of the nodes to be explored, where the priority is determined by the cost of the path from the root node to the current node. It is guaranteed to find the least costly path, but it can be memory intensive, as it needs to store all the nodes in the search space.

1. Start with the root node and add it to the priority queue with a cost of 0.
2. While the priority queue is not empty:
    a) Remove the node with the lowest cost from the priority queue.

b) If the node is a goal node, return the path to it.

c) Otherwise, expand the node and add its children to the priority queue with their respective cumulative costs.

Local Uniform Cost Search

This algorithm is a variation of uniform cost search that is used to find the least costly path from a given node to a goal node, but just considering the local neighborhood of the given node. For each selected node, the next selected node is the one with the lowest cost among its neighbors. It is not guaranteed to find the least costly path, but it can be more efficient than uniform cost search in cases where the search space is large and the goal node is close to the given node.

1. Start with the given node.
2. While the goal node has not been reached, or a maximum number of iterations has not been reached:
   a) Evaluate the neighbors of the current node and select the one with the lowest cost.
   b) Move to the selected neighbor and repeat the process.

3.3.7. Dijkstra's Algorithm

This algorithm is a specific implementation of uniform cost search that is used to find the shortest path from a source node to all other nodes in a weighted graph. It works by maintaining a priority queue of nodes to be explored, where the priority is determined by the cumulative cost from the source node to the current node. The algorithm iteratively expands the node with the lowest cumulative cost and updates the costs of its neighbors until all nodes have been explored. It is guaranteed to find the shortest path from the source node to all other nodes, but it can be memory intensive, as it needs to store all the nodes in the search space.

1. Start with the source node and add it to the priority queue with a cost of 0. Every other node is added to the priority queue with a cost of infinity.
2. While the priority queue is not empty (it is not empty until all nodes have been explored):
   a) Remove the node with the lowest cumulative cost from the priority queue.
   b) For each neighbor of the current node:
      1) Calculate the cumulative cost to reach the neighbor through the current node.
      2) If this cumulative cost is less than the previously known cost for that neighbor, update the cost and modify it in the priority queue.

It should be noted that Dijkstra's algorithm is the same as UCS, but it simply does not stop when it finds a goal node, but rather continues to explore until all nodes have been explored. This is because Dijkstra's algorithm is designed to find

the shortest path from a source node to all other nodes in the graph, while UCS is designed to find the least costly path from a root node to a specific goal node. Therefore, Dijkstra's algorithm is more general than UCS, but also more computationally intensive.

3.4. Informed Search

Informed search algorithms are those that have additional information about the search space. They use this information to guide the search and improve its efficiency. The additional information is usually in the form of a heuristic function, which estimates the cost of reaching the goal from a given node.

3.4.1. Greedy Best First Search

This algorithm uses a priority queue to keep track of the nodes to be explored, where the priority is determined by the heuristic function. It always expands the node that is estimated to be closest to the goal.

1. Start with the root node and add it to the priority queue with a priority determined by the heuristic function.
2. While the priority queue is not empty:
   a) Remove the node with the lowest heuristic value from the priority queue.
   b) If the node is a goal node, return the path to it.
   c) Otherwise, expand the node and add its children to the priority queue with their respective heuristic values.

The efficiency of Greedy Best First Search depends on the quality of the heuristic function. The main drawback of this algorithm is that it is not guaranteed to find the optimal solution, as it does not consider the actual cost of the path, but only its estimated cost to the goal.

3.4.2. A* Search

This algorithm is a combination of uniform cost search and greedy best first search. It uses a priority queue to keep track of the nodes to be explored, where the priority is determined by:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the actual cost from the root node to the current node n, and $h(n)$ is the heuristic estimate of the cost from n to the goal. Two important properties of the heuristic function are:

- Admissible: It never overestimates the cost to reach the goal. It can be expressed as:
  $$h(n) \leqslant h^*(n) \qquad \forall n \in V$$
  where $h^*(n)$ is the true cost from node n to the goal. If the heuristic is admissible, A* Search is guaranteed to find the optimal solution.

- Consistent (or monotonic): For every node n and every successor n' of n, the estimated cost of reaching the goal from n is no greater than the cost of getting from n to n' plus the estimated cost of reaching the goal from n'. Formally, this can be expressed as:

$$h(n) \leqslant c(n, n') + h(n')$$

where $c(n, n')$ is the actual cost of moving from node n to node n'. If the heuristic is consistent, it is also admissible, and A* Search is guaranteed to find the optimal solution. In addition, if the heuristic is consistent, A* Search is more efficient, as it does not need to revisit nodes that have already been explored.

Therefore, the A* Search algorithm is:

1. Start with the root node and add it to the priority queue with a priority determined by $f(n) = g(n) + h(n)$. All other nodes are added to the priority queue with a priority of infinity.
2. While the priority queue is not empty:
   a) Remove the node with the lowest $f(n)$ value from the priority queue.
   b) If the node is a goal node, return the path to it.
   c) Otherwise, expand the node. For each child of the current node:
      1) Calculate $g(n')$ as $g(n) + c(n, n')$, where $c(n, n')$ is the actual cost of moving from node n to node n'. If that new $g(n')$ value is less than the previously known $g(n')$ value for that child, update $g(n')$ and calculate $f(n') = g(n') + h(n')$.

3.5. Local Search

Local search algorithms are those that operate on a single solution and try to find a better solution by making small changes to it. They are often used in optimization problems. Therefore, the aim is finding a local optimum, not necessarily a global one.

3.5.1. Hill Climbing

This algorithm starts with an arbitrary solution and iteratively makes small changes to it, accepting the change if it results in a better solution. The process continues until no further improvements can be made (or a maximum number of iterations is reached).

1. Start with an arbitrary solution.
2. Generate the neighbors of the current solution by making small changes to it.
3. Evaluate the neighbors and select the one with the best value.
4. If the best neighbor is better than the current solution, move to that neighbor and repeat the process. Otherwise (if no neighbor is better), return the current solution as a local optimum.

3.5.2. Simulated Annealing

This algorithm is a variation of hill climbing that allows for occasional moves to worse solutions in order to escape local optima. It is inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to remove defects and improve its properties. The algorithm works as follows:

1. Start with an arbitrary solution, an initial temperature $T \geqslant 0$, and a cooling rate α (where $0 < \alpha < 1$).
2. Generate a neighbor of the current solution by making a small change to it.
3. Evaluate the neighbor and calculate the change in the objective function ΔE (difference between the neighbor's value and the current solution's value).
4. If $\Delta E < 0$ (the neighbor is better), move to that neighbor. Otherwise, move to the neighbor with a probability of $e^{-\Delta E/T}$. Initially, with a high temperature, the algorithm is more likely to accept worse solutions, allowing it to explore the search space more broadly. As the temperature decreases, the algorithm becomes less likely to accept worse solutions, focusing more on local improvements.
5. Update the temperature by multiplying it by the cooling rate: $T = \alpha T$.
6. Repeat the process until a stopping criterion is met (e.g., a maximum number of iterations, a minimum temperature, or a satisfactory solution).

This algorithm doesn't necessarily find the optimal solution, but it can avoid being stuck in local optima, especially if the cooling schedule is well-designed.

4. Machine Learning Methodics

Once data has been collected, preprocessed and analyzed, the next step is to apply a machine learning model to it, in order to extract useful information and make predictions. Models are trained and, afterwards, they are evaluated to check their performance. Therefore, the whole dataset is usually split into three subsets:

- *Training set* (70-90 %): the subset of data used to train the model. The model learns from this data by adjusting its parameters to minimize the error between its predictions and the actual values.

- *Validation set* (10-15 %): the subset of data used to tune the hyperparameters of the model. Hyperparameters are parameters that are not learned from the data, but rather set by the user.

- *Test set* (10-30 %): the subset of data used to evaluate the performance of the trained model. The model makes predictions on this data, and its performance is measured using various metrics, such as accuracy, precision, recall, F1 score, etc.

There are three main types of machine learning: There are three main types of machine learning:

- Unsupervised Learning: The machine is trained on unlabeled data, where it must find patterns and relationships on its own.

  It usually does not perform well on complex tasks, but it is perfect for outlier detection, as it can identify data points that do not fit the general pattern of the data.

- Supervised Learning: The machine is trained on labeled data, where the correct output is provided for each input.

  Labelling data can be time-consuming and expensive, and enterprises often use CAPTCHAs (Completely Automated Public Turing test to tell Computers and Humans Apart) to outsource this task to humans. While detecting if you are a human or a bot by analyzing how you click on images, you are actually helping to label data for machine learning models.

- Reinforcement Learning: The machine learns by interacting with an environment and receiving feedback in the form of rewards or penalties based on its actions.

4.1. Classification

Classification is a supervised learning task where the goal is to assign a label to an input based on its features. For example, given an email, the task is to classify it as spam or not spam. A really common classifier is the *Binary Classifier*, which is used to classify data into two classes, whether yes or no, true or false, spam or not spam, etc. A *threshold* is used to determine the decision boundary between the two classes. If the predicted probability of an input belonging to a certain class is above the threshold, it is classified as that class; otherwise, it is classified as the other class.

When working with a binary classifier, it is important to distinguish between the actual class and the predicted class. This leads to four possible outcomes:

- True Positive (TP): The model correctly predicts the positive class.
- True Negative (TN): The model correctly predicts the negative class.
- False Positive (FP): The model incorrectly predicts the positive class when it is actually negative.
- False Negative (FN): The model incorrectly predicts the negative class when it is actually positive.

This results in the following terms:

- Actual Positive (AP): The actual class is positive. $AP = TP + FN$.
- Actual Negative (AN): The actual class is negative. $AN = TN + FP$.
- Predicted Positive (PP): The predicted class is positive. $PP = TP + FP$.
- Predicted Negative (PN): The predicted class is negative. $PN = TN + FN$.

For a classifier with n classes, this is all summarized in the *confusion matrix*, which is a $n \times n$ matrix where the rows represent the predicted classes and the columns represent the actual classes[1], and the (i, j)-th entry represents the number/percentage of instances where the predicted class is i and the actual class is j. Therefore, the diagonal entries represent the correct predictions, while the off-diagonal entries represent the incorrect predictions. For a binary classifier, the confusion matrix is represented in the Table 4.1.

4.1.1. Evaluation Metrics

The first metric that comes to mind to evaluate the performance of a classifier is *accuracy*:

- Accuracy: the proportion of *correct predictions* out of the total number of *predictions*. The question is: *How many predictions were correct?*

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.1}$$

[1]This is the most common way to represent the confusion matrix, but it can also be represented with the rows and columns swapped.

Tabla 4.1: Confusion Matrix for a Binary Classifier.

		Actual Values		
		Positive	Negative	
Predicted Values	Positive	TP	FP	PP
	Negative	FN	TN	PN
		AP	AN	

This metric can however be misleading when the classes are imbalanced, meaning that one class is much more frequent than the other. For example, if 95 % of the data belongs to the negative class and only 5 % belongs to the positive class, a model that always predicts negative will have an accuracy of 95 %, which seems good, but it is actually a terrible model because it never predicts the positive class correctly. Therefore, in cases of imbalanced classes, it is better to use other metrics, as the following.

- Precision: the proportion of *true positive predictions* out of the total number of *positive predictions*. Also called *positive predictive value*. The question is: *If predected positive, how likely is it to be actually positive?*

$$\text{Precision} = PPV = P(AP \mid PP) = \frac{TP}{TP + FP} \tag{4.2}$$

- Recall: the proportion of *true positive predictions* out of the total number of *actual positives*. Also called *sensitivity* or *true positive rate*. The question is: *If actually positive, how likely is it to be predicted positive?*

$$\text{Recall} = TPR = P(TP \mid AP) = \frac{TP}{TP + FN} \tag{4.3}$$

Even though a theoretically perfect model would have both precision and recall equal to 1, this is in practice very difficult to achieve, and there is usually a trade-off between them. For example, if we set a very low threshold for a binary classifier, it will predict positive for almost all instances, which will result in a high recall but a low precision. On the other hand, if we set a very high threshold, it will predict positive for very few instances, which will result in a high precision but a low recall. Therefore, it is important to find the right balance between precision and recall depending on the specific problem and the costs associated with false positives and false negatives:

- In medical diagnosis, a false negative (failing to detect a disease) can be much more costly than a false positive (incorrectly diagnosing a healthy person), so we would want to prioritize recall over precision.
- In email spam detection, a false positive (marking a legitimate email as spam) can be more costly than a false negative (failing to detect a spam email), so we would want to prioritize precision over recall.

As precision and recall are important metrics for the positive class, they also have their opposite metrics for the negative class, which are also important to evaluate the performance of a classifier

- Negative Predictive Value (NPV): the proportion of *true negative predictions* out of the total number of *negative predictions*. The question is: *If predicted negative, how likely is it to be actually negative?*

$$NPV = P(AN \mid PN) = \frac{TN}{TN + FN} \tag{4.4}$$

- Specificity: the proportion of *true negative predictions* out of the total number of *actual negatives*. Also called *true negative rate*. The question is: *If actually negative, how likely is it to be predicted negative?*

$$\text{Specificity} = TNR = P(TN \mid AN) = \frac{TN}{TN + FP} \tag{4.5}$$

Out of this four metrics (precision, recall, NPV and specificity), we can derive their four complementary metrics:

- False Positive Rate (FPR): the proportion of *false positive predictions* out of the total number of *actual negatives*. Also called *false alarm rate*.

$$FPR = P(PP \mid AN) = \frac{FP}{FP + TN} = 1 - \text{TNR} \tag{4.6}$$

- False Discovery Rate (FDR): the proportion of *false positive predictions* out of the total number of *positive predictions*.

$$FDR = P(AN \mid PP) = \frac{FP}{TP + FP} = 1 - \text{PPV} \tag{4.7}$$

- False Omission Rate (FOR): the proportion of *false negative predictions* out of the total number of *negative predictions*.

$$FOR = P(AP \mid PN) = \frac{FN}{TN + FN} = 1 - \text{NPV} \tag{4.8}$$

- False Negative Rate (FNR): the proportion of *false negative predictions* out of the total number of *actual positives*. Also called *miss rate*.

$$FNR = P(PN \mid AP) = \frac{FN}{TP + FN} = 1 - \text{TPR} \tag{4.9}$$

Given that trade-offs between precision and recall exist, it is important to have single metrics that combines both of them. The following are some of the most common metrics that combine the metrics we have seen so far:

- F1 Score: the harmonic mean of precision and recall.

$$F1 = \frac{2}{\frac{1}{\text{PPV}} + \frac{1}{\text{TPR}}} = \frac{2 \cdot \text{PPV} \cdot \text{TPR}}{\text{PPV} + \text{TPR}} \tag{4.10}$$

- Fowlkes-Mallows index: the geometric mean of precision and recall.

$$FM = \sqrt{\text{PPV} \cdot \text{TPR}} \tag{4.11}$$

- Matthews Correlation Coefficient (MMC): a correlation coefficient between the actual and predicted classes.

$$MCC = \sqrt{\text{TPR} \cdot \text{TNR} \cdot \text{PPV} \cdot \text{NPV}} - \sqrt{\text{FNR} \cdot \text{FPR} \cdot \text{FDR} \cdot \text{FOR}} \tag{4.12}$$

  It is the geometric mean of the basic metrics (TPR, TNR, PPV and NPV) minus the geometric mean of the complementary metrics (FNR, FPR, FDR and FOR). It ranges from -1 to 1, where:

  - 1 indicates a perfect classifier.
  - 0 indicates a random classifier.
  - -1 indicates a completely wrong classifier.

  It is the most informative metric for evaluating the performance of a binary classifier, especially when the classes are imbalanced, as it the only one that takes into account the whole confusion matrix.

Other metrics that are commonly used to evaluate the performance of a classifier are:

- Prevalence: the proportion of actual positives in the dataset.

$$\text{Prevalence} = P(AP) = \frac{AP}{AP + AN} \tag{4.13}$$

- Balanced Accuracy: the average of the true positive rate and the true negative rate.

$$\text{Balanced Accuracy} = \frac{\text{TPR} + \text{TNR}}{2} \tag{4.14}$$

  It is a better metric than accuracy when the classes are imbalanced, as it takes into account both the true positive rate and the true negative rate.

- Positive Likelihood Ratio (LR_+): the ratio of the true positive rate to the false positive rate.

$$LR_+ = \frac{\text{TPR}}{\text{FPR}} \tag{4.15}$$

  It indicates how much more likely a positive test result is to be found in an actual positive case than in an actual negative case. The higher the LR_+, the better the test is at identifying actual positives.

- Negative Likelihood Ratio (LR_-): the ratio of the false negative rate to the true negative rate.

$$LR_- = \frac{\text{FNR}}{\text{TNR}} \tag{4.16}$$

  It indicates how much more likely a negative test result is to be found in an actual positive case than in an actual negative case. The lower the LR_-, the better the test is at identifying actual negatives.

- Diagnostic Odds Ratio (DOR): the ratio of the positive likelihood ratio to the negative likelihood ratio.

$$DOR = \frac{LR_+}{LR_-} \tag{4.17}$$

  The higher the DOR, the better the test is at discriminating between actual positives and actual negatives.

- Informedness: the difference between the true positive rate and the false positive rate.

$$\text{Informedness} = \text{TPR} - \text{FPR} = \text{TPR} + \text{TNR} - 1 \tag{4.18}$$

- Markedness: the difference between the positive predictive value and the false discovery rate.

$$\text{Markedness} = \text{PPV} - \text{FDR} = \text{PPV} + \text{NPV} - 1 \tag{4.19}$$

- Prevalence Threshold (PT): the prevalence at which the positive likelihood ratio equals the negative likelihood ratio.

$$PT = \frac{\sqrt{TPR \cdot FPR} - FPR}{TPR - FPR} \tag{4.20}$$

Lastly, an important curve used to evaluate the performance of a binary classifier is the *Receiver Operating Characteristic (ROC) curve*, which plots the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings using the same dataset. The area under the ROC curve (AUC) is a common metric used to summarize the performance of a binary classifier, where a higher AUC indicates a better performance.

- An ideal classifier would have all the points in the ROC curve at the top-left corner, which corresponds to a TPR of 1 and a FPR of 0, resulting in an AUC of 1.
- The bigger the AUC, the better the performance of the classifier.

4.2. Error and Uncertainty

When evaluating the performance of a machine learning model, it is important to take into account the error and uncertainty associated with the predictions. The different types of errors are:

- Blunders: errors caused by human carelessness, such as accidentally deleting data, mislabeling data, or using the wrong algorithm. These errors can be easily avoided by being careful and double-checking the work.
- Random Errors: uncontrollable errors that occur due to fluctuations in variables. They can be:
  - *Environmental*: caused by changes in the environment, such as temperature, humidity, etc.

  - *Observational*: when the observer's judgment leads to random innacuracies.

- Systematic Errors: errors that are identifiable and can be corrected. They can be:

  - *Instrumental*: caused by a flaw in the measurement instrument, such as a miscalibrated sensor.
  - *Theoretical*: caused by a flaw in the theoretical model, such as an incorrect assumption.
  - *Environmental*: caused by a consistent change in the environment, such as a change in the temperature that affects the measurements.
  - *Observational*: when the observer does not read the measurement instrument correctly, such as reading the wrong value on a scale.

However, an important trade-off is that no model is perfect, and there will always be some error associated with the predictions, as the data themselves are not perfect, and the model is just an approximation of reality. Therefore, it is important to evaluate the performance of a model using appropriate metrics that take into account the error and uncertainty associated with the predictions. Depending on the task, different metrics are used to evaluate the performance of a model.

- In classification tasks (prediction of discrete labels), the error measurement has already been covered in the previous section, where we have seen various metrics to evaluate the performance of a classifier.

- In regression tasks (prediction of continuous values), the error is usually measured using the following metrics, where the predicted values are denoted by $\hat{y}_i$ and the actual values are denoted by y_i:

  - Mean Absolute Error (MAE): the average of the absolute differences between the predicted values and the actual values.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{4.21}$$

    As an advantage, it is less sensitive to outliers than the mean squared error.

  - Mean Squared Error (MSE): the average of the squared differences between the predicted values and the actual values.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{4.22}$$

    As an disadvantage, it is more sensitive to outliers than the MAE, and it can be difficult to interpret the error in the original units of the data, as it is in squared units.

- Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{MSE} \tag{4.23}$$

- Mean Absolute Percentage Error (MAPE): the average of the absolute percentage differences between the predicted values and the actual values.

$$MAPE = \frac{1}{n}\left(\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|\right) \cdot 100\,\% \tag{4.24}$$

  As an advantage, it is easy to interpret the error in terms of percentage, which can be more meaningful in some contexts. As a disadvantage, it can be undefined when any of the actual values are zero, and it can be heavily influenced by small actual values, leading to very large percentage errors.

4.3. Overfitting and Underfitting

Two common problems that can occur when training a machine learning model are overfitting and underfitting:

- Overfitting: occurs when a model learns the training data too well, *including the noise and outliers*, which results in a model that performs extremely well on the training data but poorly on unseen data, as it has lost the ability to generalize to new data.

  Some common techniques to prevent overfitting are resampling methods (which will be covered later), early stopping or increasing the complexity of the model.

- Underfitting: occurs when a model is too simple to capture the underlying patterns in the data, which results in a model that performs poorly on both the training data and unseen data.

  Some common techniques to prevent underfitting are increasing the complexity of the model, increasing the training duration or resampling methods.

Both overfitting and underfitting are directly related to the bias-variance trade-off, which is a fundamental concept in machine learning.

- Bias: the error introduced by approximating a real-world problem making incorrect assumptions. An algorithm with high bias may think that it is correctly modeling the data, but it is actually oversimplifying the problem.

  A machine learning model with high bias pays very little attention to the training data and oversimplifies the model, which can lead to underfitting.

- Variance: the error introduced by the model's sensitivity to small fluctuations in the training data. An algorithm with high variance may fit the training data very well, but it may not generalize well to unseen data, as it has learned the noise and outliers in the training data.

  A machine learning model with high variance pays a lot of attention to the training data and does not generalize well, which can lead to overfitting.

The bias-variance trade-off states that minimizing one of these two sources of error typically increases the other. In other words, as we increase the complexity of a model, its bias decreases but its variance increases, and vice versa. Therefore, it is important to find the right balance between bias and variance to achieve good generalization performance on unseen data.

4.3.1. No Free Lunch Theorem

In order to find that correct balance between bias and variance, we need to choose the right model for our data. However, the *No Free Lunch Theorem* states that there is no one model (or parameter setting) that works best for all problems. In other words, the performance of a model is highly dependent on the specific problem and the data it is applied to. Therefore, it is important to try different models and parameter settings to find the one that works best for our specific problem and data.

In order to find the best model and parameter settings, we can use a brute-force approach, where we try all possible combinations of models and parameters. However, the *curse of dimensionality* makes this approach infeasible, as the number of combinations grows exponentially with the number of models and parameters. Therefore, a more practical approach is to use a *trial and error* method, where we try a subset of models and parameter settings based on our intuition and experience, and then evaluate their performance using appropriate metrics to find the best one for our specific problem and data.

4.4. Resampling Methods

Resampling methods are techniques used to, from the available data, create new samples that can be used to train and evaluate machine learning models. These methods are particularly useful when the dataset is small, as they allow us to make the most out of the available data.

4.4.1. Data Augmentation

Data augmentation is a technique used to increase the size of the training dataset by creating new samples from the existing data. This can be done by applying various transformations to the data, such as rotation, scaling, flipping, etc. Data augmentation is commonly used in image classification tasks, where it can help to improve the performance of the model by providing it with more diverse examples to learn from.

4.4.2. Cross-Validation

Cross-validation is a technique used to evaluate the performance of a machine learning model by partitioning the data into multiple subsets, training the model on some of the subsets and evaluating it on the remaining subsets. Multiple techniques have been developed for cross-validation, which can be broadly categorized into two types: exhaustive and non-exhaustive cross-validation techniques.

Exhaustive Cross-Validation

Exhaustive cross-validation techniques involve evaluating the model on all possible combinations of training and test sets. These techniques include:

- Leave-One-Out: the model is trained on all the data except for one instance, which is used as the test set. This process is repeated for each instance in the dataset, and the performance of the model is averaged over all iterations.

  If there are n instances in the dataset, this technique requires training the model n times, which can be computationally expensive for large datasets.

- Leave-P-Out: the model is trained on all the data except for p instances, which are used as the test set. This process is repeated for all possible combinations of p instances, and the performance of the model is averaged over all iterations.

  If there are n instances in the dataset, this technique requires training the model $\binom{n}{p}$ times, which can be computationally expensive for large datasets and large values of p.

Since exhaustive cross-validation techniques can be computationally expensive, especially for large datasets, more practical approaches have been developed.

Non-Exhaustive Cross-Validation

Non-exhaustive cross-validation techniques involve evaluating the model on a subset of all possible combinations of training and test sets. These techniques include:

- K-Fold Cross-Validation: the data is partitioned into k subsets (or folds), where the model is trained on $k-1$ folds and evaluated on the remaining fold. This process is repeated k times, with each fold used as the test set once, and the performance of the model is averaged over all iterations.

  With k folds, this technique requires training the model k times, which is more computationally efficient than exhaustive cross-validation techniques, especially for large datasets.

- Holdout Method: the data is randomly split into a training set and a test set, where the model is trained on the training set and evaluated on the test set. This method is simple and computationally efficient, but it can be sensitive to how the data is split, as it may not be representative of the overall dataset.

- Repeated Random Subsampling: the data is randomly split into a training set and a test set multiple times, where the model is trained on the training set and evaluated on the test set for each split, and the performance of the model is averaged over all iterations. This repetition helps to mitigate the problems of the holdout method.

- Bootstrapping: the data is randomly sampled *with replacement* to create multiple training sets, where the model is trained on each training set and evaluated on the remaining data (called the out-of-bag samples), and the performance of the model is averaged over all iterations.

4.5. K-Nearest Neighbors (KNN)

Even though a whole chapter could be dedicated to classification algorithms, we will focus right now on one of the most basic and widely used algorithms: K-Nearest Neighbors (KNN). KNN is a simple and intuitive machine learning algorithm used for different tasks, such as classification and regression. Its main idea is that, given a new data point, it will behave similarly to the data points that are closest to it in the feature space. The process is:

- Choosing a k value, which represents the number of nearest neighbors to consider.
- Calculating the distance between the new data point and all the data points in the training set, using a distance metric such as Euclidean distance, Manhattan distance, etc.
- Sort the distances and select the k nearest neighbors.
- Depending on the task, different strategies are used to make the prediction:
  - Classification: the predicted class is the majority class among the k nearest neighbors.

    It is true that other approaches would also make sense, such as just stating the probability of the new data point belonging to each class based on the proportion of neighbors belonging to each class, or weighting the contribution of each neighbor based on their distance to the new data point, giving more weight to closer neighbors. However, they are not used.
  - Regression: the predicted value is the average of the values of the k nearest neighbors.
  - Outliers Detection: in order to detect if a value is an outlier, we can calculate the distance between the new data point and its k nearest neighbors, and if the average distance (weighted or not) is above a certain threshold, we can classify it as an outlier.

5. Genetic Algorithms

Genetic algorithms are a class of optimization algorithms inspired by the process of natural selection. They are used to find approximate solutions to optimization and search problems. The paralelism between the process of natural selection and the genetic algorithm is described in the following concepts:

- Chromosome: A representation of a solution to the problem. It is a data structure that encodes all the parameters of the solution.

  They are often represented as strings of bits, but they can also be represented as arrays of numbers or other data structures depending on the problem being solved.

- Gene: A part of a chromosome that represents a specific parameter or feature of the solution.

- Alele: A specific value of a gene. They can be binary (0 or 1), discrete (a finite set of values), or continuous (a range of values).

A genetic algorithm typically consists of the following steps:

1. Initialization: A population of chromosomes is generated randomly or based on some heuristic. The size of the population will be constant throughout the algorithm.

2. Selection: The fitness of each chromosome is evaluated using a fitness function that measures how well the solution represented by the chromosome solves the problem. Chromosomes with higher fitness are more likely to be selected for reproduction.

3. Crossover: Pairs of selected chromosomes are combined to create offspring. This is done by exchanging parts of the chromosomes between the pairs.

4. Mutation: With a small probability, random changes are made to the genes of the offspring to introduce variation into the population.

5. Replacement: The new generation of chromosomes (offspring) replaces the old generation, and the process repeats from the selection step until a stopping criterion is met:

   - A solution with a satisfactory fitness level is found.
   - A maximum number of generations is reached.

- The population has converged (i.e., there is little variation in the chromosomes).

During the following sections, we will explore some of these steps in more detail.

5.1. Crossover

Crossover is a genetic operator used to combine the genetic information of two parent chromosomes to generate a new offspring chromosome. There are several types of crossover methods, including:

- Order-Based Crossover (OX2): Fixed positions are selected from the first parent, their values are copied to the offspring in the same positions, and the holes are filled with the corresponding values from the second parent.

Parent 1	**A**	**A**	B	**B**	**A**	A	C	C	**A**	A
Parent 2	C	C	A	B	A	B	B	A	A	A
Offspring	**A**	**A**	A	**B**	**A**	B	B	A	**A**	A

Tabla 5.1: Example of Order-Based Crossover (OX2), where the fixed positions are 0-1, 3-4 and 8.

- Position-Based Crossover (POS): Fixed positions are selected from the first parent. The values in those positions are copied at the start of the offspring, and the remaining values are filled with the start of the second parent.

Parent 1	**A**	**A**	B	**B**	**A**	A	C	C	**A**	A
Parent 2	C	C	A	B	A	B	B	A	A	A
Offspring	**A**	**A**	**B**	**A**	**A**	C	C	A	B	A

Tabla 5.2: Example of Position-Based Crossover (POS), where the fixed positions are 0-1, 3-4 and 8.

- Voting Crossover (VX):

  For each gene, the value that appears most frequently among the parents is selected for the offspring. In case of a tie, one of the values is randomly selected.

  This makes it a more general crossover method that can be applied to more than two parents, as it can take into account the values of all the parents when determining the value of each gene in the offspring.

Parent 1	A	A	B	B	A	A	C	C	A	A
Parent 2	C	C	A	B	A	B	B	A	A	A
Parent 3	A	A	A	B	A	A	C	C	A	A
Offspring	A	A	A	B	A	A	C	C	A	A

Tabla 5.3: Example of Voting Crossover (VX) with three parents.

- Maximal Preservative Crossover (MPX):

  An interval of genes is selected from the first parent and copied to the offspring (in the same positions). The remaining genes are filled with the values from the second parent, starting from the start of the interval.

Parent 1	A	A	B	B	A	A	C	C	A	A
Parent 2	C	C	**A**	**B**	**A**	**B**	B	A	A	A
Offspring	**A**	**B**	B	B	A	A	C	C	**A**	**B**

Tabla 5.4: Example of Maximal Preservative Crossover (MPX), where the selected interval is 2-7.

  It should be noted that, if the interval selected in the first parent is too short, there may not be enough genes to fill the offspring. This should be controlled, as in some cases, chromosomes should not vary in length.

- Alternating Positions Crossover (AP):

  The offspring is created by alternating the genes of the two parents, but with two important constraints:

  - Positions should be kept. If a gene is in the position i in offspring, it should be in the same position i in the parent from which it was taken.
  - Genes can't be repeated in a chromosome.

  This results in the following procedure to create the offspring:

  1. The first gene of the offspring is taken from the first position of the first parent.
  2. For every gene in the offspring, the next gene is taken from the other parent. If that gene is already in the offspring, the order of the parents is switched, and the next gene for that position is chosen from the other parent.
     - It that gene is also already in the offspring (which is not likely), then a mutation has to be performed, as there is no way to fill that position with a gene from the parents without repeating genes. This mutation can be done by randomly selecting a gene that is not already in the offspring and placing it in that position.
  3. This process is repeated until the offspring is complete.

  It should be noted that, if the parents are very similar, there may not be enough genes to fill the offspring without repeating genes. This should be controlled, as in some cases, chromosomes should not vary in length.

5.2. Mutation

Mutation is a genetic operator used to maintain genetic diversity within a population of chromosomes. It introduces random changes to the genes of offspring

chromosomes, which can help prevent premature convergence to local optima. There are several types of mutation types, as:

- Silent Mutation: A mutation that does not change the genotype of the chromosome. For example, if a gene is represented as a binary string, a silent mutation would be changing a 0 to another 0 or a 1 to another 1.
- Neutral Mutation: A mutation that changes the genotype of the chromosome but does not affect the phenotype (the observable characteristics of the solution).
- Conditional Mutation: A mutation that also changes the phenotype, but is still considered a valid solution to the problem.
- Letal Mutation: A mutation that results in a non-viable solution (e.g., a solution that violates constraints of the problem).

Depending on the problem being solved, different mutation methods will result in letal mutations, which should be avoided since they do not contribute to the search for optimal solutions. The choice of mutation method and the mutation rate (the probability of mutation occurring) can have a significant impact on the performance of a genetic algorithm. A high mutation rate can lead to excessive randomness and slow convergence, while a low mutation rate may not introduce enough diversity into the population, leading to premature convergence. There are different basic methods to perform mutation, such as:

- Deletion: A segment of the chromosome is removed.
- Insertion: A segment of the chromosome is selected and inserted into another position in the chromosome.
- Duplication: A segment of the chromosome is duplicated and inserted into another position in the chromosome.
- Inversion: A segment of the chromosome is reversed.

Using those, we can create more complex mutation methods, such as:

- Scramble Mutation: A segment of the chromosome is selected and its genes are randomly shuffled.
- Swap Mutation: Two genes in the chromosome are selected and their positions are swapped.

It is true that some of these mutation methods can change the length of the chromosome, which may not be desirable in certain problems where a fixed-length representation is required. This should be taken into account, as they could end up being letal mutations if the resulting chromosome is not a valid solution to the problem.

5.3. Selection

Selection is the process of choosing which chromosomes from the current population will be used to create the next generation. The selection process is based on the fitness of each chromosome, which is determined by a fitness function that evaluates how well the solution represented by the chromosome solves the problem. A correct balance between exploration and exploitation is crucial for the success of a genetic algorithm:

- Exploration: The ability of the algorithm to search through a wide range of solutions in the solution space. This is important to avoid getting stuck in local optima and to find the global optimum.
- Exploitation: The ability of the algorithm to focus on promising areas of the solution space that have already been explored. This is important to refine and improve solutions that are already good.

Two important concepts in selection are:

- Selection Pressure: The degree to which the selection process favors fitter chromosomes over less fit ones. High selection pressure can lead to faster convergence but may also increase the risk of premature convergence to local optima.
- Diversity: The variety of different chromosomes in the population. Maintaining diversity is important to ensure that the algorithm explores a wide range of solutions and does not get stuck in local optima.

There are several selection methods, including:

- Elitist Selection: The best (for instance, 10 %) chromosome(s) from the current population are directly copied to the next generation (without being subject to crossover or mutation), ensuring that the best solution is preserved.
- Steady State Selection: Only a small number of chromosomes (e.g., 30 %) are replaced in each generation, while the rest of the population remains unchanged. This allows for a gradual evolution of the population over time.

  It should be noted that this approach is just a variation of elitist selection, where the portion of the "elite" is much larger than in the standard elitist selection.
- Tournament Selection: Fixed k chromosomes are randomly selected from the population, and the one with the highest fitness is chosen for reproduction. This process is repeated until the desired number of chromosomes is selected.
- Roulette Wheel Selection: Each chromosome is assigned a probability of selection proportional to its fitness. If f_i is the fitness of chromosome i, then:

$$P(i) = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

where N is the total number of chromosomes in the population. It is seen as a roulette wheel where each chromosome occupies a portion of the wheel proportional to its fitness, and the wheel is spun to select chromosomes for reproduction. The process is repeated until the desired number of chromosomes is selected.

- Rank-Based Selection: Chromosomes are ranked based on their fitness, and selection probabilities are assigned based on their rank rather than their absolute fitness values. The roulette wheel selection method is then applied to these new probabilities. This can help to maintain diversity in the population and prevent premature convergence.

It should however be noted that the selection process may combine different methods, such as using elitist selection to preserve the best solutions while also applying tournament selection to select the remaining chromosomes needed for reproduction and reaching the desired population size for the next generation.

6. Clustering and Classification

Even though supervised learning is the most common type of machine learning, unsupervised learning is the most common type of learning in the world. Therefore, not only supervised but also unsupervised learning is important to understand. The two main techniques used for unsupervised and supervised learning are, respectively, clustering and classification. In this chapter, we will discuss these two techniques in detail.

6.1. Clustering

Clustering is a technique used to group similar data points together. During the following sections, we will discuss some of the most common clustering algorithms, including K-Means, Hierarchical Clustering, and DBSCAN.

6.1.1. K-Means

K-Means is a popular clustering algorithm that partitions the data into k clusters. The exact division of the data into clusters is determined by different algorithms that soon will be discussed; however, it has the following general disadvantages:

- The number of clusters k must be specified beforehand, which can be difficult to determine. In order to determine the optimal number of clusters, techniques such as the Elbow Method can be used.

  The Elbow Method involves plotting the WCSS (Within-Cluster Sum of Squares) against the number of clusters k. The WCSS measures the total distance between each point and the centroid of its assigned cluster. This plot is always decreasing as k increases, but the goal is to find the point where the rate of decrease sharply changes, resembling an "elbow". This point indicates the optimal number of clusters, as adding more clusters beyond this point does not significantly reduce the WCSS.

- K-Means only works well with spherical clusters (if the euclidean distance is used) or cubical clusters (if the Manhattan distance is used). It may not perform well with clusters of arbitrary shapes.

- K-Means is sensitive to the initial placement of centroids, which can lead to different results on different runs. If the initial centroids are chosen within the same cluster, the algorithm may converge to a local minimum. Different initialization methods, such as K-Means++, can be used to improve the convergence and stability of the algorithm.

It has several specific applications, such as image compression:

- In image compression, K-Means can be used to reduce the number of colors in an image. Each pixel's color is treated as a data point in a three-dimensional space (representing the RGB color values). K-Means clusters these colors into k groups, and each pixel is then assigned the color of its corresponding cluster centroid. This results in a compressed image with fewer colors while maintaining visual similarity to the original.

Some of the algorithms used to determine the clusters in K-Means include:

Lloyd's Algorithm

Lloyd's Algorithm is the most common algorithm used for K-Means clustering. It works as follows:

1. Initialize k centroids randomly. They can be selected randomly from the data points or generated randomly within the data space.

2. Assign each data point to the nearest centroid, forming k clusters.

$$C_i = \{x_j : \|x_j - \mu_i\|^2 \leqslant \|x_j - \mu_k\|^2, \forall k, 1 \leqslant k \leqslant K\}$$

   where C_i is the set of data points assigned to cluster i, x_j is a data point, μ_i is the centroid of cluster i, and $\|\cdot\|$ denotes a distance (usually Euclidean).

3. Update the centroids by calculating the mean of the data points in each cluster:

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

4. Repeat steps 2 and 3 until convergence, which occurs when the centroids do not change significantly or a maximum number of iterations is reached.

This algorithm is efficient and works well for many datasets, but it can converge to a local minimum, which is why the initialization of centroids is crucial.

MacQueen's Algorithm

MacQueen's Algorithm is a different approach to K-Means clustering that updates the centroids incrementally as data points are assigned to clusters. It works as follows:

1. Select the first k data points as the initial centroids.

2. For each data point, assign it to the nearest centroid and update the centroid's position immediately after the assignment. The update is done using the following formula:

$$\mu_{\text{new}} = \mu_{\text{old}} + \frac{x_j - \mu_{\text{old}}}{n_i + 1}$$

   where μ_{old} is the current centroid, x_j is the new data point being assigned, and n_i is the number of data points currently assigned to that centroid before the

new assignment. This formula updates the centroid by moving it towards the new data point, taking into account the number of points already assigned to that cluster.

3. Repeat this process for all data points until convergence, which occurs when the centroids do not change significantly or a maximum number of iterations is reached.

This algorithm can be more efficient than Lloyd's Algorithm for large datasets, as it updates the centroids incrementally rather than waiting for all data points to be assigned before updating.

Jenks-Caspall Algorithm

The Jenks-Caspall Algorithm is a method used for optimizing the classification of data into clusters, particularly in the context of choropleth maps. Its aim is to use the *natural breaks* in the data to determine the optimal number of clusters and their boundaries, achieving two main objectives:

- Minimize the variance within each cluster, ensuring that data points within the same cluster are as similar as possible.
- Maximize the variance between clusters, ensuring that different clusters are as distinct as possible.

It works in two main cycles:

1. Re-iterative Cycling: Use the previously explained algorithms (Lloyd's or MacQueen's) to determine the clusters and their centroids. This process may be repeated multiple times to refine the clusters and improve the classification.
2. Forced Cycling: This step involves forcing the algorithm to consider different cluster boundaries by adjusting the centroids and reassigning data points. This can help to escape local minima and find a more optimal clustering solution.

Otsu's method

Otsu's method is a technique used for image thresholding, which can be considered a form of clustering in the context of image processing. It works by finding the optimal threshold that separates the pixel intensity values into two classes (foreground and background) while maximizing the between-class variance.

X-Means

X-Means is an extension of the K-Means algorithm that automatically determines the optimal number of clusters. It works by starting with a small number of clusters ($k = 2$) and then iteratively splitting clusters based on a criterion such as the Bayesian Information Criterion (BIC)[1]. The BIC is a statistical measure that:

- Penalizes the complexity of the model (number of clusters) to avoid overfitting.

[1]The WCSS can't be used, as it always decreases.

- Rewards the goodness of fit of the model to the data.

For each cluster, two options are evaluated:

- Keep the cluster as it is.
- Split the cluster into two sub-clusters and evaluate the BIC for both the original and the split clusters.

If the BIC improves after splitting, the cluster is split; otherwise, it remains unchanged.

K-Means++

K-Means++ is an initialization method for the K-Means algorithm that aims to improve the convergence and stability of the algorithm by selecting initial centroids in a more strategic way. The process works as follows:

1. Randomly select the first centroid from the data points.
2. For each subsequent centroid, select a data point with a probability proportional to the square of the distance (d^2) from the nearest existing centroid. This means that data points that are farther away from the existing centroids are more likely to be selected as new centroids.
3. Repeat this process until k centroids have been selected.

This solves an important problem of the K-Means algorithm, which is the sensitivity to the initial placement of centroids. If two centroids are initialized within the same cluster, the algorithm may converge to a local minimum. By using K-Means++, the initial centroids are more likely to be spread out across the data space, leading to better convergence and improved clustering results.

K-Medians

K-Medians is a variant of the K-Means algorithm that uses the median instead of the mean to update the centroids. This makes it more robust to outliers, as the median is less affected by extreme values than the mean.

K-Medoids

K-Medoids (also known as PAM - Partitioning Around Medoids) is another variant of the K-Means algorithm that uses actual data points (medoids) as the centroids instead of calculating the mean or median. This can be more robust to outliers and can work better with non-spherical clusters. The algorithm works as follows:

1. Initialize k medoids randomly from the data points.
2. Assign each data point to the nearest medoid, forming k clusters.
3. In each cluster, select the data point that minimizes the total distance to all other points in the cluster as the new medoid.

This process is repeated until the medoids do not change, indicating convergence.

6.1.2. Hierarchical Clustering

Hierarchical Clustering is a method of cluster analysis that builds a hierarchy of clusters. It can be divided into two main types:

- Agglomerative Clustering: This is a bottom-up approach where each data point starts as its own cluster, and pairs of clusters are merged as one moves up the hierarchy. The merging process continues until all data points are in a single cluster or until a specified number of clusters is reached.
- Divisive Clustering: This is a top-down approach where all data points start in a single cluster, and the cluster is recursively split into smaller clusters until each data point is in its own cluster or until a specified number of clusters is reached.

The structure of the hierarchy can be represented as a *dendrogram*, which is a tree-like diagram that shows the arrangement of the clusters formed at each step of the algorithm. The height of the branches in the dendrogram represents the distance or dissimilarity between clusters, allowing for the visualization of how clusters are merged or split. This method is sensitive to outliers, as they can significantly affect the structure of the hierarchy and the resulting clusters.

Since divisive clustering is only seldomly used, in the following sections different linkage criteria for agglomerative clustering will be discussed. In each step of the agglomerative clustering process, the two clusters that are closest to each other are merged (so the number of clusters decreases by one). The distance between clusters can be defined in different ways, leading to different linkage criteria.

Single Linkage

In single linkage, the distance between two clusters is defined as the minimum distance between any two points in the two clusters.

$$d_{\text{single}}(A, B) = \min_{a \in A, b \in B} d(a, b)$$

This method can lead to the *chaining effect*, where clusters can be formed by a series of close points, even if the overall clusters are not well separated. The reason for this is that single linkage focuses on the closest points between clusters, which can result in elongated and irregularly shaped clusters.

Complete Linkage

In complete linkage, the distance between two clusters is defined as the maximum distance between any two points in the two clusters.

$$d_{\text{complete}}(A, B) = \max_{a \in A, b \in B} d(a, b)$$

This method tends to produce more compact and spherical clusters, as it considers the farthest points between clusters. However, it can be sensitive to outliers, as a single outlier can significantly increase the distance between clusters.

Average Linkage

In average linkage, the distance between two clusters is not decided by just two points, but rather by all the points in the clusters. There are different specific ways to define the average distance:

- Unweighted Average Linkage: The distance between two clusters is defined as the average distance between all pairs of points in the two clusters.

$$d_{\text{unweighted}}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{a \in A,\ b \in B} d(a, b)$$

- Weighted Average Linkage: The distance between two clusters is defined as the average distance between all pairs of points in the two clusters, but with weights assigned to the distances based on the size of the clusters.

$$d_{\text{weighted}}(A, B) = \frac{|A|}{|A| + |B|} d_{\text{unweighted}}(A, B) + \frac{|B|}{|A| + |B|} d_{\text{unweighted}}(B, A)$$

- Centroid Method Unweighted Pair-Group Method using Centroids: The distance between two clusters is defined as the distance between their centroids. The centroid of a cluster is the mean of all the points in the cluster.

$$d_{\text{centroid}}(A, B) = d(\mu_A, \mu_B)$$

  where μ_A and μ_B are the centroids of clusters A and B, respectively.

- Median Method Weighted Pair-Group Method using Medians: The distance between two clusters is defined as the distance between their medians. The median of a cluster is the median of all the points in the cluster.

$$d_{\text{median}}(A, B) = d(\text{median}(A), \text{median}(B))$$

- Ward's Method: This method minimizes the total within-cluster variance. The distance between two clusters is defined as the increase in the total within-cluster variance that would result from merging the two clusters.

$$d_{\text{ward}}(A, B) = \frac{|A| \cdot |B|}{|A| + |B|} d^2_{\mu_A, \mu_B}$$

  This method tends to produce clusters of similar size and can be more effective in identifying compact clusters. The reason for this is that Ward's method focuses on minimizing the variance within clusters, which encourages the formation of clusters that are more homogeneous and compact.

This method provides a compromise between single and complete linkage, as it considers the overall distance between clusters rather than just the closest or farthest points. It can produce clusters that are more balanced in terms of shape and size.

6.1.3. DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together data points that are closely packed together, while marking points that lie alone in low-density regions as outliers. It works based on two parameters:

- Epsilon (ε): This parameter defines the radius of the neighborhood around a data point. It determines how close points need to be to each other to be considered part of the same cluster.

- `MinPts`: This parameter defines the minimum number of points required to form a dense region.

The algorithm classifies data points into three categories:

- Core Points: These are points that have at least `MinPts` points within their ε-neighborhood. Core points are the central points of clusters.

- Border Points: These are points that have fewer than `MinPts` points within their ε-neighborhood but are within the ε-neighborhood of a core point. Border points are on the edge of clusters.

- Noise Points: These are points that do not have enough neighboring points to be classified as core points and are not within the ε-neighborhood of any core point. Noise points are considered outliers.

The DBSCAN algorithm works as follows:

1. For each unvisited point, check if it is a core point by counting the number of points within its ε-neighborhood.
    - If the point is a core point, create a new cluster. All points that are density-reachable from this core point (i.e., all points that can be reached by traversing through core points) are added to the cluster.
    - If the point is a border point, it is not computed. It will be considered when its associated core point is processed.
    - If the point is a noise point, it is marked as an outlier and is not included in any cluster.
2. Repeat this process until all points have been visited.

It has some advantages over other clustering algorithms, as it can find clusters of arbitrary shapes and is robust to outliers. However, it can be sensitive to the choice of parameters (ε and `MinPts`), and it may not perform well with varying densities of clusters.

6.1.4. OPTICS

OPTICS (Ordering Points To Identify the Clustering Structure) is an extension of the DBSCAN algorithm that addresses some of its limitations, particularly in handling clusters of varying densities. It uses two parameters similar to DBSCAN:

- Epsilon (ε): This parameter defines the maximum radius of the neighborhood around a data point. It is used to avoid a core point to search for neighbors indefinitely, which can lead to performance issues.
- `MinPts`: This parameter defines the minimum number of points required to form a dense region, also similar to DBSCAN.

The OPTICS algorithm introduces two new concepts to better capture the clustering structure:

- Core Distance: This is the distance from a point to its `MinPts`-th nearest neighbor. If a point has fewer than `MinPts` neighbors within ε, its core distance is undefined (or set to ∞).

$$d_{\text{core}}(p) = \begin{cases} \text{distance to the } \texttt{MinPts} - \text{th nearest neighbor of } p & \text{if } |N_\varepsilon(p)| \geqslant \texttt{MinPts} \\ \infty & \text{otherwise} \end{cases}$$

- Reachability Distance: This is the distance from a point p to another point o that is reachable from p. It is defined as:

$$d_{\text{reachability}}(p, o) = \text{máx}(d_{\text{core}}(p), d(p, o))$$

  The maximum function ensures that the reachability distance reflects both the core distance of p and the actual distance between p and o. If both points are close in a dense region, the reachability distance is set to the core distance, which helps to identify clusters of varying densities (as every core point in the same cluster will have similar core distances). If the points are farther apart, the reachability distance will reflect the actual distance between them, which helps to distinguish between different clusters.

The OPTICS algorithm uses two different lists to keep track of the points:

- An ordered list of points, which maintains the order in which points are processed. It contains all the points in the dataset and their corresponding reachability distances when they are added to the list. The order of points in this list does not change once they are added.
- A priority queue of points to be processed, which is ordered by their reachability distance. Points with lower reachability distances have higher priority and are processed first.

Using those two lists, the OPTICS algorithm works as follows:

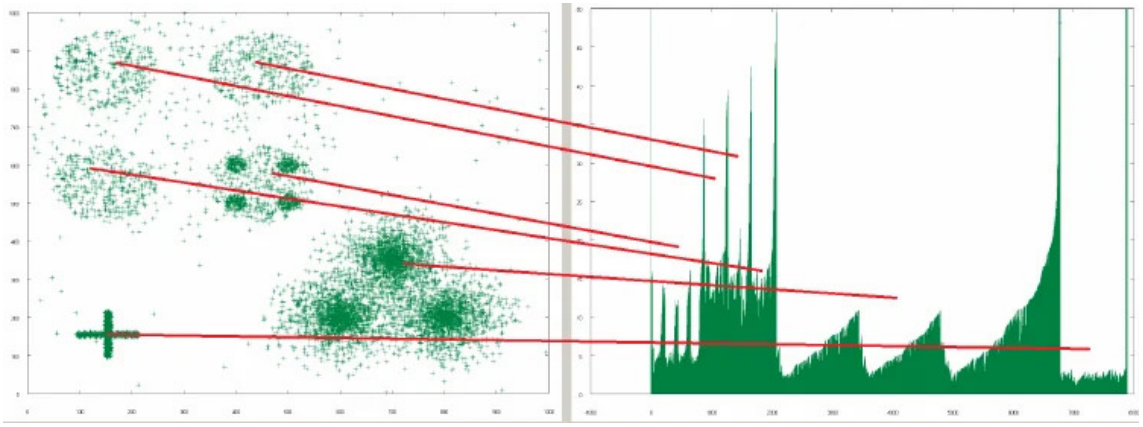

Figura 6.1: Example of a reachability plot. The valleys represent clusters, while the peaks represent noise points.

1. A not processed point is selected and added to the ordered list with a reachability distance of ∞ (as a second point would be needed to calculate the reachability distance). If it is a core point, all its `MinPts` neighbors are added to the priority queue with their reachability distances from the selected core point.

    - The first point in the priority queue is selected and added to the ordered list. If it is a core point, all its `MinPts` neighbors are added to the priority queue with their reachability distances from the selected core point (if they were already in the priority queue, their reachability distance is updated if the new reachability distance is smaller).

2. That process is repeated until the priority queue is empty, and then the first step is repeated again. This process continues until all points have been processed and added to the ordered list.

After processing all points, a reachability plot can be created, which is a visual representation of the reachability distances of the points in the order they were processed.

- X-axis: The points ordered by the order in which they were processed (i.e., the order in which they were added to the ordered list).
- Y-axis: The reachability distance of each point.

In the reachability plot, clusters can be identified as valleys (regions of low reachability distance) separated by peaks (regions of high reachability distance). A really dense cluster will have a long valley with low reachability distances, while a less dense cluster will have a shorter valley with higher reachability distances. Noise points will appear as peaks in the reachability plot, as they have high reachability distances. An example of a reachability plot is shown in the Figure 6.1.

This method allows for the identification of clusters of varying densities, as the reachability distances will reflect the density of the clusters. It has however a big disadvantage, which is that it has a subjective component in the interpretation of the reachability plot, as it can be difficult to determine where the valleys and peaks are, especially in cases where the clusters are not well defined.

6.2. Classification

Classification is a technique used to assign labels to data points based on their features. Even though its generlities and how could its performance be evaluated where explained in detail in Section 4.1, during the following sections, we will discuss some of the most common classification algorithms, including K-Nearest Neighbors, Decision Trees, and Support Vector Machines.

6.2.1. Logistic Regression

Logistic Regression is a statistical model that is used for binary classification tasks. It models the probability of a data point belonging to a particular class using the logistic function (also known as the sigmoid function). The logistic function maps any real-valued number into the $[0, 1]$ interval, which can be interpreted as a probability. The logistic regression model can be expressed as follows:

$$P(z) = \frac{1}{1 + e^{-z}}$$

where $P(z)$ is the probability of the data point belonging to the positive class, and z is a linear combination of the input features. The linear combination can be expressed as:

$$z = w^t \cdot x + b$$

where w is the weight vector, x is the input feature vector, and b is the bias term. The weights and bias are learned from the training data using an optimization algorithm such as gradient descent.

Once the model is trained, it can be used to predict the class of new data points by calculating the probability $P(z)$ and applying a threshold (usually 0,5) to determine the predicted class:

- If $P(z) \geqslant 0{,}5$, the data point is classified as belonging to the positive class.
- If $P(z) < 0{,}5$, the data point is classified as belonging to the negative class.

6.2.2. K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a simple and intuitive algorithm that was already explained in Section 4.5. As it was stated there, it can be used for classification tasks, selecting the most common class among the k nearest neighbors of a data point.

6.2.3. Support Vector Machines (SVM)

Support Vector Machines (SVM) is a powerful classification algorithm that finds the optimal hyperplane that separates the data points of different classes in the feature space. Among all the possible hyperplanes that can separate the classes, SVM selects the one that maximizes the margin, which is the distance between the hyperplane and the nearest data points from each class (called support vectors). This approach helps to improve the generalization of the model and reduce the risk of overfitting.

Observación. It should be noted that only the support vectors are relevant for determining the position of the hyperplane, as they are the closest points to the decision boundary. The other data points do not affect the position of the hyperplane, which is why SVM can be effective even with a small number of points.

The equation of the optimal hyperplane can be expressed as:

$$w^t \cdot x + b = 0$$

where:

- w is the weight vector that is perpendicular to the hyperplane and determines its orientation.
- x is the input feature vector.
- b is the bias term that shifts the hyperplane away from the origin.

Two classes of data points are considered, a positive class (labeled as $+1$) and a negative class (labeled as -1). The support vectors are the data points that satisfy the following conditions:

- For the positive class: $w^t \cdot x + b = 1$
- For the negative class: $w^t \cdot x + b = -1$

The classification of a new data point x can be determined by evaluating the equation of the hyperplane:

- If $w^t \cdot x + b \geqslant 1$, the data point is classified as belonging to the positive class.
- If $w^t \cdot x + b \leqslant -1$, the data point is classified as belonging to the negative class.
- If $-1 < w^t \cdot x + b < 1$, the data point is classified as being on the margin, and it may be classified as either class depending on the specific implementation of the SVM algorithm.

In order to determine the margin γ, lets consider a point x_- support vector from the negative class and the corresponding point x_+ that lies on the margin of the positive class. Since w is perpendicular to the hyperplane, the following equation holds:

$$x_+ = x_- + \gamma \frac{w}{\|w\|}$$

Since x_+ lies on the margin of the positive class, it satisfies the equation $w^t \cdot x_+ + b = 1$. Taking these two equations together, we can derive the following equation:

$$\begin{aligned} w^t \cdot \left(x_- + \gamma \frac{w}{\|w\|} \right) + b &= 1 \\ w^t \cdot x_- + \gamma \frac{w^t \cdot w}{\|w\|} + b &= 1 \\ w^t \cdot x_- + \gamma \|w\| + b &= 1 \end{aligned}$$

Taking into account that x_- is a support vector from the negative class, it satisfies the equation $w^t \cdot x_- + b = -1$. Substituting this into the previous equation, we get:

$$-1 + \gamma\|w\| = 1 \Longrightarrow \gamma = \frac{2}{\|w\|}$$

Taking into account that the margin γ should be maximized, the optimal hyperplane can be found. An important parameter of the SVM algorithm is the regularization parameter C, which controls the trade-off between maximizing the margin and minimizing the classification error on the training data.

- A small value of C allows for a larger margin, even if it means that some data points are misclassified. This can lead to better generalization on unseen data, as the model is less likely to overfit the training data.
- A large value of C forces the model to classify all training data points correctly, which can lead to a smaller margin and potentially overfitting the training data.

The selection of the regularization parameter C is crucial for the performance of the SVM model. Lastly, it should be noted that SVM can be extended to handle non-linearly separable data by using kernel functions, which implicitly map the input features into a higher-dimensional space where a linear separation is possible.

6.2.4. Decision Trees

Decision Trees are a type of classification algorithm that uses a tree-like model of decisions and their possible consequences. The structure of a decision tree consists of nodes and branches:

- Root Node: This is the topmost node of the tree, which represents the entire dataset. It is the starting point for making predictions.
- Internal Nodes: These nodes represent the features or attributes used to split the data. Each internal node corresponds to a specific feature and a threshold value that is used to divide the data into subsets.
- Leaf Nodes: These nodes represent the final output of the tree, which is the predicted class label for the data points that reach that node.

The decision tree is constructed by recursively splitting the dataset based on the features that provide the best separation of the classes. The splitting process continues until a stopping criterion is met, such as:

- All data points in a node belong to the same class.
- The maximum depth of the tree is reached.
- The minimum number of data points required to split a node is not met.

As happened with kNN, the decision tree algorithm can be used for both classification and regression tasks.

- In classification tasks, the predicted class label for a data point is determined by the majority class of the data points that reach the corresponding leaf node.
- In regression tasks, the predicted value for a data point is determined by the average value of the data points that reach the corresponding leaf node.

The decision tree algorithm has several advantages, such as being easy to interpret and visualize, handling both numerical and categorical data, and being able to capture non-linear relationships between features. However, it can also be prone to overfitting and the resulting tree is usually instable, as small changes in the data can lead to a completely different tree structure.

Splitting Criteria

Different criteria can be used to determine the best feature and threshold for splitting the data at each internal node of the decision tree. Some of the most common splitting criteria include:

- Entropy: This criterion measures the impurity of a node based on the distribution of class labels. The goal is to minimize the entropy after the split. It is calculated as:

$$S(X) = -\sum_{x \in K} p(x) \log_{|K|}(p(x))$$

  where:

  - K is the set of all possible class labels.
  - $S(X)$ is the entropy of the node.
  - $p(x)$ is the proportion of data points in the node that belong to class x.

  It should be noted that the logarithm is taken with base $|K|$, and $0 \cdot \log_{|K|}(0)$ is defined as 0. The minus sign is used to ensure that the entropy is a non-negative value, as the logarithm of a probability is always negative or zero. The lower the entropy, the more pure the node is, meaning that it contains data points that mostly belong to a single class.

- Gini Impurity: This criterion measures the impurity of a node based on the probability of misclassifying a randomly chosen data point from the node. The goal is to minimize the Gini impurity after the split. It is calculated as:

$$G(X) = 1 - \sum_{x \in K} p(x)^2$$

  where:

  - K is the set of all possible class labels.
  - $G(X)$ is the Gini impurity of the node.
  - $p(x)$ is the proportion of data points in the node that belong to class x.

When the Gini impurity of every child node is known, the Gini impurity of the parent node can be calculated as the weighted average of the Gini impurity of the child nodes:

$$G_{\text{parent}} = \sum_i \frac{|X_i|}{|X|} G(X_i)$$

Observación. Both entropy and Gini impurity are commonly used splitting criteria for decision trees, and they often yield similar results. However, Gini impurity is computationally more efficient than entropy, as it does not involve calculating logarithms. As a result, Gini impurity is often preferred in practice, especially when dealing with large datasets.

Information Gain

Information Gain is a measure used to evaluate the effectiveness of a feature in splitting the data. It is calculated as the difference between the entropy of the parent node and the weighted average entropy of the child nodes after the split. The formula for Information Gain is:

$$IG(X, A) = S(X) - \sum_{v \in V_A} \frac{|X_v|}{|X|} S(X_v)$$

where:

- X is the set of data points in the parent node.
- A is the feature being evaluated for the split.
- V_A is the set of all possible values for feature A.
- X_v is the subset of data points in the parent node that have value v for feature A.

A higher Information Gain indicates that the feature is more effective in splitting the data, as it results in a greater reduction in entropy. Therefore, when constructing a decision tree, the feature with the highest Information Gain is typically selected for splitting at each internal node.

ID 3 Algorithm

The ID 3 (Iterative Dichotomiser 3) algorithm is a specific implementation of the decision tree algorithm. Its aim is to construct a decision tree where the leaf nodes are as pure as possible. The algorithm works as follows:

1. Start with the entire dataset as the root node of the tree.
2. Calculate the entropy for each division according to each unused feature in the dataset and select the feature whose division results in the lowest entropy to split the data at the current node.
3. Create child nodes for each possible value of the selected feature and partition the data accordingly.

4. Repeat steps 2 and 3 recursively for each child node until one of the following stopping criteria is met:

    - All data points in the node belong to the same class (i.e., the node is pure).
    - There are no more features to split on.
    - The maximum depth of the tree is reached.

C 4.5 Algorithm

The C 4.5 algorithm is an extension of the ID 3 algorithm that addresses some of its limitations. It introduces several improvements in order to better handle continuous features, missing values and overfitting. It uses some base cases:

1. If all data points in the node belong to the same class, the node is made a leaf node with that class label.

2. If no feature provides a positive Information Gain, the node is made a leaf node with the majority class label of the data points in that node.

3. If a previously unseen class is encountered (i.e., a class that is not present in the training data), the higher up node is made a leaf node with the majority class label of the data points in that node.

Its main differences with the ID 3 algorithm are:

- It uses the Information Gain instead of the Entropy to select the best feature for splitting. This can lead to better splits, as Information Gain takes into account the reduction in entropy after the split, rather than just the entropy of the parent node.

- It checks the bases cases before calculating the Gain Ratio, which can help to reduce the depth of the tree and improve its generalization.

Bagging

This technique uses bootstraping (explained in page **??**) to create multiple subsets of the training data, and then trains a separate decision tree on each subset. The final prediction depends on the task:

- For classification tasks, the final prediction is determined by majority voting among the predictions of the individual trees.

- For regression tasks, the final prediction is determined by averaging the predictions of the individual trees.

Random Forests

Random Forests is an extension of the decision trees that combines two techniques:

- Bagging: As explained in the previous section, it creates multiple subsets of the training data and trains a separate decision tree on each subset.
- Random Feature Selection: When splitting a node during the construction of the tree, instead of considering all features, it randomly selects a subset of features and only considers those for splitting.

This has several advantages, as it helps to reduce the correlation between the individual trees, which can lead to better generalization and improved performance on unseen data. Additionally, it can handle large datasets with high dimensionality and is less prone to overfitting compared to a single decision tree.

7. Regression

During this chapter, we will cover regression, which is a fundamental technique in machine learning used to model and analyze the relationships between variables. We will explore various methods for regression, including linear regression, regularized regression techniques, and outlier detection methods. Additionally, we will discuss classification methods that can be adapted for regression tasks.

7.1. Distances

Before diving into regression, it's essential to understand the concept of distances, as they play a crucial role in many regression algorithms. Different types of distances can be used to measure the similarity or dissimilarity between data points, which can affect the performance of regression models. Four conditions for a function to be considered a distance are:

1. Non-negativity: $d(x, y) \geqslant 0$ for all x, y.
2. Identity of indiscernibles: $d(x, y) = 0$ if and only if $x = y$.
3. Symmetry: $d(x, y) = d(y, x)$ for all x, y.
4. Triangle inequality: $d(x, z) \leqslant d(x, y) + d(y, z)$ for all x, y, z.

Some common types of distances include:

- Euclidean Distance (L^2): The straight-line distance between two points in a multi-dimensional space, based on the Pythagorean theorem.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- Minkowski Distance (L^p): A generalization of the Euclidean distance that includes other distance metrics as special cases.

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

- Manhattan Distance (L^1): The sum of the absolute differences between the coordinates of two points, often used in grid-like path scenarios.

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- Max-Distance: This distance measures the maximum difference between the coordinates of two points, also known as Chebyshev distance.

$$d(x, y) = \max_i |x_i - y_i|$$

- Min-Distance: It measures the minimum difference between the coordinates of two points.

$$d(x, y) = \min_i |x_i - y_i|$$

  It should be noted that, strictly speaking, the Min-Distance does not satisfy all the conditions to be considered a true distance. However, it is still a useful metric in certain contexts.

- Cosine Similarity: Measures the cosine of the angle between two vectors, which is a measure of their orientation rather than their magnitude. It is commonly used in text analysis and high-dimensional spaces.

$$d(x, y) = \cos(\theta) = \frac{x \cdot y}{\|x\|\|y\|}$$

Some specific distances for particular types of data include:

- Hamming Distance: Used for discrete objects or strings, it counts the number of positions at which the corresponding symbols are different.

$$d(x, y) = \sum_{i=1}^{n} \delta(x_i, y_i)$$

  where $\delta(x_i, y_i) = 1$ if $x_i \neq y_i$ and 0 otherwise. For instance, the Hamming distance between "kitten" and "sitting" is 3.

- Mahalanobis Distance: A distance measure that accounts for the correlations between variables and the scale of the data, making it useful for multivariate data.

$$d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

  where Σ is the covariance matrix of the data:

$$\Sigma = E[(X - \mu)(X - \mu)^T]$$

  It takes into account the variance of each variable and the covariance between variables, making it particularly effective for identifying outliers in multivariate data.

7.1.1. Similarity Measures

In addition to distances, similarity measures are also crucial in regression and other machine learning tasks. They quantify how alike a dataset is, which can be particularly useful in clustering and classification tasks. Some common similarity measures include:

- Entropy: A measure of uncertainty or randomness in a dataset, often used in decision tree algorithms to determine the best splits.

$$S(X) = -\sum_{x \in X} P(x) \log P(x)$$

where $P(x)$ is the probability of occurrence of the value x in the dataset. Depending on the base of the logarithm used, it is measured in bits (base 2), nats (base e), or bans (base 10).

- Negentropy: A measure of the degree of order or structure in a dataset, defined as the difference between the maximum possible entropy and the actual entropy of the dataset.

$$N(X) = S_{\max} - S(X)$$

where $S_{\max}$ is the maximum entropy of the dataset, which occurs when all outcomes are equally likely.

- Kullback-Leibler Divergence (Relative Entropy): A measure of how one probability distribution $Q(X)$ diverges from a reference probability distribution $P(X)$.

$$D_{KL}(P \| Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$$

It quantifies the amount of information lost when Q is used to approximate P.

- Cross-Entropy: A measure of the entropy of an approximation $Q(X)$ relative to the true distribution $P(X)$, often used as a loss function in classification problems.

$$H(P, Q) = -\sum_{x \in X} P(x) \log Q(x)$$

It measures the average number of bits needed to identify an event from a set of possibilities, given a predicted distribution Q instead of the true distribution P. It should be noted that:

$$H(P, Q) = S(P) + D_{KL}(P \| Q)$$

- Mutual Information: A measure of the mutual dependence between two variables, quantifying the amount of information obtained about one variable through the other.

$$MI(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \left(\frac{P(x, y)}{P(x) P(y)} \right)$$

If $MI(X; Y) = 0$, it indicates that the variables are independent, while a higher value indicates a stronger dependency between the variables, and therefore more shared information.

7.2. Dimensionality Reduction

Incoming data often has a high number of features, mathematically represented as a high-dimensional space. However, many of these features may be redundant or irrelevant, leading to the curse of dimensionality. Dimensionality reduction techniques aim to reduce the number of features while preserving the essential structure and information in the data. This can improve model performance, reduce overfitting, and enhance interpretability. There are two main types of dimensionality reduction techniques:

- Linear Techniques: These methods assume that the data lies on or near a linear subspace of the original feature space. Examples include Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA).
- Non-Linear Techniques: These methods can capture complex, non-linear relationships in the data. Examples include t-Distributed Stochastic Neighbor Embedding (t-SNE) and Uniform Manifold Approximation and Projection (UMAP).

7.2.1. PCA (Principal Component Analysis)

PCA is a widely used linear dimensionality reduction technique that transforms the original features into a new set of uncorrelated features called principal components. It works as follows:

1. Standardize the data to have a mean of zero and a standard deviation of one.
2. Compute the covariance matrix of the standardized data.
3. Calculate the eigenvalues and eigenvectors of the covariance matrix.
4. Sort the eigenvalues in descending order and select the top k eigenvectors corresponding to the largest eigenvalues. These eigenvectors form the new feature space.
5. Project the original data onto the new feature space to obtain the reduced-dimensional representation.

The principal components capture the directions of maximum variance in the data, allowing for effective dimensionality reduction while retaining most of the information. The number of principal components to retain can be determined based on the desired level of explained variance. First components capture the most variance, while later components capture less.

Eigenfaces

Eigenfaces is a practical application of PCA in the field of computer vision, specifically for facial recognition. The principal components are called "eigenfaces", and when represented as images, they resemble ghostly faces. By projecting facial images onto the eigenface space, one can efficiently recognize and reconstruct faces based on their principal components.

7.2.2. Factor Analysis (FA)

Factor Analysis is another linear dimensionality reduction technique that models the observed variables as linear combinations of latent factors (basic structures of the data), plus some error terms. It assumes that the data follows a Gaussian distribution and aims to identify the underlying structure in the data. FA is particularly useful when the number of observed variables is large, and the eigenvalues and eigenvectors of the covariance matrix are computationally expensive to calculate.

7.2.3. FastICA

Fast Independent Component Analysis (FastICA) is an optimization algorithm for performing Independent Component Analysis (ICA). It is designed to find a linear transformation that maximizes the statistical independence of the components. FastICA is particularly effective for separating mixed signals, such as in blind source separation problems, where the goal is to recover original source signals from observed mixtures.

7.2.4. SVD (Singular Value Decomposition)

SVD is a general matrix factorization technique that decomposes a matrix into three matrices:

$$A = U\Sigma V^T$$

where:

- U is a rotation matrix.
- Σ is a diagonal matrix containing the singular values, which represent the scaling factors of each component.
- V^T is another rotation matrix.

SVD can be used for dimensionality reduction by retaining only the top k singular values and their corresponding singular vectors, effectively capturing the most significant features of the data.

7.2.5. t-SNE (t-Distributed Stochastic Neighbor Embedding)

t-SNE is a non-linear dimensionality reduction technique that is particularly effective for visualizing high-dimensional data in 2D or 3D. As the main advantage, t-SNE focuses on preserving local structures in the data, revealing clusters and patterns that linear techniques might miss. It works by converting the high-dimensional distances between data points into probabilities and then minimizing the Kullback-Leibler divergence between the original and the reduced-dimensional representations.

1. Construct a probability distribution over pairs of high-dimensional objects in such a way that similar objects have a higher probability of being picked, while dissimilar points have a lower probability.

2. Define a similar probability distribution over the points in the low-dimensional map.

3. Map the high-dimensional data to a low-dimensional space by minimizing the Kullback-Leibler divergence between the two distributions using gradient descent.

Observación. The SNE algorithm, which is the predecessor of t-SNE, had a significant drawback in that it could not effectively handle the so-called "crowding problem", where points that are close together in high-dimensional space could end up being mapped far apart in the low-dimensional representation. t-SNE addresses this issue by using a Student's t-distribution to model the similarities in the low-dimensional space instead of a Gaussian distribution, which allows it to better preserve local structures and reveal clusters in the data.

7.2.6. UMAP (Uniform Manifold Approximation and Projection)

UMAP is an extension of t-SNE that is designed to be faster and more scalable while still preserving both:

- Local Structure: Like t-SNE, UMAP focuses on preserving the local relationships between data points, ensuring that similar points remain close together in the reduced-dimensional space.
- Global Structure: UMAP also aims to preserve the global structure of the data, which allows it to maintain the overall shape and distribution of the data in the low-dimensional representation.

7.3. Linear Regression

Linear regression is the main core of this chapter, and it is a fundamental technique for modeling the relationship between a dependent variable (y) and one or more independent variables (x). The goal of linear regression is to find the best-fitting line (or hyperplane in higher dimensions) that minimizes the distance between the observed data points and the predicted values from the model. The basic form of a simple linear regression model can be expressed as:

$$y = \beta x + \epsilon$$

If x is a vector of independent variables, then β is a vector of coefficients, and β_j represents the change in y for a one-unit change in x_j, holding all other variables constant. The error term ϵ captures the variability in y that cannot be explained by the linear relationship with x.

Some different measures of the error in linear regression include:

- Mean Absolute Error (MAE): The average of the absolute differences between observed and predicted values.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Mean Squared Error (MSE): The average of the squared differences between observed and predicted values.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE): The square root of the MSE, providing an error measure in the same units as the dependent variable.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- Mean Absolute Percentage Error (MAPE): The average of the absolute percentage differences between observed and predicted values.

$$MAPE = \frac{1}{n}\left(\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|\right) \cdot 100\,\%$$

- R-squared (R^2): A statistical measure that represents the proportion of the variance in the dependent variable that is explained by the independent variables in the model.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

  An R^2 value of 1 indicates that the model perfectly explains the variance in the dependent variable, while an R^2 value of 0 indicates that the model does not explain any of the variance.

In order to determine the best-fitting line, we can use different methods, such as:

- Ordinary Least Squares (OLS): This method minimizes the sum of the squared differences between observed and predicted values, effectively minimizing the MSE. The loss function for OLS can be expressed as:

$$L = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Lasso Linear Regression (L1): If the dimensionality reduction of the data was not sufficient, overfitting can occur. Lasso regression is a regularization technique that can help mitigate overfitting by adding a penalty term to the loss function based on the absolute values of the coefficients. This may lead to some unimportant coefficients being reduced to zero, effectively performing *feature selection* and reducing the complexity of the model. The loss function for Lasso can be expressed as:

$$L = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$

where λ is the regularization parameter that controls the strength of the penalty.

- Ridge Linear Regression (L2): Ridge regression is another regularization technique that adds a penalty term based on the squared values of the coefficients. Coefficients are *shrunk towards zero* but not exactly zero, which can help to reduce the variance of the model without eliminating any features entirely. This method is particularly effective in cases where there is *multicollinearity* among the independent variables, as it can help to reduce the variance of the coefficient estimates. The loss function for Ridge can be expressed as:

$$L = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

where λ is the regularization parameter that controls the strength of the penalty.

- Elastic Net Linear Regression: Elastic Net is a regularization technique that combines both Lasso and Ridge regression. It adds a penalty term that is a linear combination of the L1 and L2 penalties, allowing for both feature selection and coefficient shrinkage. The loss function for Elastic Net can be expressed as:

$$L = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^{p} |\beta_j| + \lambda_2 \sum_{j=1}^{p} \beta_j^2$$

where λ_1 and λ_2 are the regularization parameters that control the strength of the L1 and L2 penalties, respectively.

Observación. The choice of regularization technique and the tuning of the regularization parameters (λ) are crucial for achieving the best performance of the regression model. Cross-validation is often used to select the optimal values for these parameters, ensuring that the model generalizes well to unseen data. In that case, it is called Elastic Net Cross-Validation (ElasticNetCV).

7.4. Outlier detection

Outliers are data points that deviate significantly from the majority of the data, and they can have a substantial impact on the performance of regression models. In order to work with outliers *without having to detect them*, we can use robust regression techniques that are less sensitive to outliers. The main robust regression technique is RANSAC (Random Sample Consensus).

- RANSAC (Random Sample Consensus): RANSAC is a robust regression algorithm that is designed to fit a model to data that may contain outliers. It works as follows:

  1. Randomly select a subset of the data points and fit a model to this subset (using one of the previous regression techniques).

2. Determine how many data points from the entire dataset fit the model within a certain tolerance (these points are called inliers).
3. Repeat the process a specified number of times, and keep the model that has the highest number of inliers.

RANSAC is particularly effective in situations where the dataset contains some outliers that could significantly affect the performance of traditional regression methods. However, some disadvantages of RANSAC include its sensitivity to the choice of parameters (such as the number of iterations and the tolerance level) and its computational cost, especially for large datasets.

However, there are also other methods for outlier detection that can be used in conjunction with regression models. They all produce a *decision boundary* that separates inliers from outliers, and they can be used to identify and remove outliers from the dataset before fitting a regression model. In the following, we will discuss some of these methods.

7.4.1. LOF (Local Outlier Factor)

LOF is an unsupervised anomaly detection method that identifies outliers based on the local density of data points. It compares the local density of a point to the local densities of its neighbors, and points that have a significantly lower density than their neighbors are considered outliers. The algorithm works as follows:

1. For each pair of data points A, B, calculate the distance $d(A, B)$ and the $k-\text{distance}(B)$, which is the distance to the k-th nearest neighbor of B. The reachability distance is defined as:

$$\text{reachability\_distance}(A, B) = \text{máx}\{k - \text{distance}(B), d(A, B)\}$$

It represents the distance from A to B while considering the local density around B.

2. For each data point A, calculate the local reachability density (LRD) as the inverse of the average reachability distance to its k nearest neighbors:

$$\text{LRD}(A) = \left(\frac{\sum_{B \in N_k(A)} \text{reachability\_distance}(A, B)}{|N_k(A)|} \right)^{-1}$$

3. Finally, calculate the LOF score for each data point A as the average ratio of the local reachability density of A to the local reachability densities of its k nearest neighbors:

$$\text{LOF}(A) = \frac{1}{|N_k(A)|} \left(\sum_{B \in N_k(A)} \text{LRD}(B) \right) \cdot \frac{1}{\text{LRD}(A)}$$

Depending of the value of the LOF score, we can determine whether a data point is an outlier or not.

- If LOF(A) ≈ 1, then A has a local density similar to its neighbors, and it is not considered an outlier.
- If LOF(A) > 1, then A has a lower local density than its neighbors, and it is considered an outlier. The higher the LOF score, the more likely it is that A is an outlier.
- If LOF(A) < 1, then A has a higher local density than its neighbors, and it is considered an inlier.

7.4.2. Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm that isolates anomalies by randomly partitioning the data using decision trees. The main idea behind Isolation Forest is that anomalies are easier to isolate than normal points, as they tend to be few and different from the majority of the data. The algorithm works as follows:

1. Randomly select a feature and a split value to partition the data into two subsets.
2. Repeat the partitioning process recursively until all data points are isolated (i.e., each point is in its own leaf node).
3. The path length from the root to a leaf node is used to determine the anomaly score for each data point. Outliers tend to have shorter path lengths, as they are isolated more quickly than normal points.

In order to have a more robust anomaly score, Isolation Forest typically builds an ensemble of trees (hence the name "forest") and averages the path lengths across all trees to compute the final anomaly score for each data point.

8. Neural Networks

Neural networks are computational models inspired by the human brain, designed to recognize patterns and solve complex problems. They consist of interconnected layers of artificial neurons that process and transmit information.

8.1. Artificial Neuron

An artificial neuron is the basic building block of a neural network. It takes multiple inputs and, after processing them, produces an output. The processing typically involves a weighted sum of the inputs followed by an activation function.

1. Inputs: The neuron receives multiple inputs, each associated with a weight that determines its importance.

2. Combination of Inputs: The neuron computes a weighted sum of the inputs, often adding a bias term to allow for more flexibility in the output. This is usually called "net input":
$$\sum_{i=1}^{n} w_i x_i + b$$

3. Activation Function: The result of the weighted sum is passed through an activation function, which introduces non-linearity into the model. This allows the network to learn complex patterns.

4. Output: The output of the activation function is the final output of the neuron.
$$y = \varphi\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

8.1.1. Activation Function

Activation functions are crucial in neural networks as they allow the model to capture non-linear relationships. They are called "activation" functions because they determine whether a neuron should be activated or not based on the input it receives. Common activation functions include:

- Identity: No real activation function. It is really used in the output layer for regression problems where the output can take any real value.
$$\varphi(x) = x$$

- Step Function: A binary function that outputs 1 or 0 depending on whether the input is above or below a certain threshold (often 0). Only the positive inputs activate the neuron.

$$\varphi(x) = \begin{cases} 1 & \text{if } x \geqslant 0 \\ 0 & \text{if } x < 0 \end{cases}$$

- Sign Function: Similar to the step function but outputs -1 or 1 depending on the sign of the input.

$$\varphi(x) = \operatorname{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \end{cases}$$

- Logistic/Sigmoid Function: A smooth, S-shaped function that maps any real-valued number into the $]0, 1[$ interval. It is commonly used in binary classification problems.

$$\varphi(x) = \frac{1}{1 + e^{-x}}$$

- Hyperbolic Tangent Function: Similar to the logistic function but maps inputs to the $]-1, 1[$ interval. It is often used in hidden layers of neural networks.

$$\varphi(x) = \tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- Softmax Function: Used in multi-class classification problems. A parameter T (temperature) can be introduced to control the smoothness of the output distribution.

$$\varphi(x_i) = \frac{e^{x_i/T}}{\sum_j e^{x_j/T}}$$

  where j ranges over all classes. It is important to note that the softmax function is typically applied to the output layer of a neural network when performing multi-class classification, as it converts the raw output scores (logits) into probabilities that sum to 1 across all classes.

- Rectified Linear Unit (ReLU): A popular activation function in deep learning.

$$\varphi(x) = \operatorname{máx}(0, x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geqslant 0 \end{cases}$$

  It is really useful in hidden layers of deep networks because it helps to mitigate the vanishing gradient problem, allowing for faster training and better performance. Some modern variants of ReLU include:

  - Parametric ReLU (PReLU): Introduces a parameter α to allow for a small slope in the negative part of the function.

$$\varphi(x) = \begin{cases} \alpha x & \text{if } x < 0 \\ x & \text{if } x \geqslant 0 \end{cases}$$

- Softplus: A smooth approximation of ReLU.

$$\varphi(x) = \ln(1 + e^x)$$

- Exponential Linear Unit (ELU): Similar to ReLU but allows for negative outputs, which can help with learning.

$$\varphi(x) = \begin{cases} \alpha(e^x - 1) & \text{if } x < 0 \\ x & \text{if } x \geqslant 0 \end{cases}$$

Appart from these, there are many other activation functions that have been proposed in the literature, each with its own advantages and disadvantages depending on the specific problem being addressed.

8.2. Neural Networks

A neural network is a collection of interconnected artificial neurons organized in layers. There are typically three types of layers in a neural network:

- Input Layer: The layer that receives the initial data.
- Hidden Layers: One or more layers that process the inputs from the previous layer and pass the results to the next layer. These layers are called "hidden" because they are not directly exposed to the input or output.

  Deep networks are simply neural networks with many hidden layers, which allow them to learn more complex representations of the data.
- Output Layer: The layer that produces the final output of the network.

In order for a neural network to learn from data and work effectively, it needs to be trained. The following steps are typically involved in the training process:

1. Architecture Design: The structure of the network is defined, including the number of layers, the number of neurons in each layer, the choice of activation functions and, specially, the network topology (how the layers are connected).
2. Initialization: The weights and biases of the network are initialized, often with small random values.
3. Propagation: The input data is fed through the network, and the output is computed.
4. Loss Calculation: A loss function is used to measure the difference between the predicted output and the actual target values.
5. Training: The error is propagated back through the network to update the weights and biases in a way that minimizes the loss.
6. Iteration: The process is repeated for multiple epochs (steps) until the network converges to a solution or reaches a predefined number of iterations.
7. Evaluation: The performance of the trained network is evaluated using a separate test dataset to ensure that it generalizes well to unseen data.

Given the importance of the training phase, we will focus deeper on it.

8.2.1. Training Process of Neural Networks

The training process of neural networks is crucial for their performance. It has two main phases: the error propagation phase and the weight update phase.

Backpropagation Phase

In this phase, the error is propagated backward through the network to compute the error made by each neuron. The error for the output layer is calculated separately (usually using a loss function as the MAE) and then propagated back to the hidden layers. The error for a neuron is computed as follows:

$$e_j = \sum_k \frac{w_{jk}}{\sum_i w_{ik}} \cdot e_k$$

where k ranges over the neurons in the next layer, w_{jk} is the weight connecting the current neuron to the k-th neuron in the next layer, i ranges over the neurons in the current layer, and e_k is the error of the k-th neuron in the next layer. This process continues until the error has been propagated back to all layers of the network.

However, an important problem with backpropagation is the vanishing gradient problem, which occurs when the gradients become very small as they are propagated back through the layers, making it difficult for the network to learn effectively. This is particularly problematic in deep networks with many layers. To mitigate this issue, various techniques have been developed, such as using different activation functions (like ReLU) or implementing skip connections.

Weight Update Phase

In this phase, the weights and biases of the network are updated based on the computed errors. It is done as follows:

$$w_{ij,\text{new}} = w_{ij,\text{old}} - \eta \cdot \Delta w_{ij}$$

where η is the learning rate, and Δw_{ij} is the change in the weight w_{ij}, which is the weight connecting the i-th neuron in the previous layer to the j-th neuron in the current layer. The specific method for calculating Δw_{ij} can vary, and there are several algorithms for updating the weights, including:

- Gradient Descent: A general optimization algorithm that minimizes the loss function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient.

  $$\Delta w_{ij} = \frac{\partial L}{\partial w_{ij}}$$

  where L is the loss function. Using the chain rule, this can in practice be computed as:

  $$\Delta w_{ij} = o_i \cdot \delta_j$$

  where:

- o_i is the output of the i-th neuron in the previous layer.
- δ_j is the error term for the j-th neuron in the current layer, which can be computed as:

$$\delta_j = e_j \cdot \varphi'(\text{net}_j)$$

where e_j is the error for the j-th neuron and $\varphi'(\text{net}_j)$ is the derivative of the activation function evaluated at the net input of the j-th neuron.

- Hebb's Rule: A simple learning rule based on the principle that neurons that fire together, wire together. The change in weight is proportional to the product of the input and output of the neuron.

$$\Delta w_{ij} = o_i \cdot o_j$$

where o_i is the output of the i-th neuron in the previous layer and o_j is the output of the j-th neuron in the current layer. Stochastic Gradient Descent (SGD): A variant of gradient descent that updates the weights using a single training example (or a small batch of examples) at a time, rather than the entire dataset. This can lead to faster convergence and better generalization, especially in large datasets, as it allows the model to escape local minima and explore the loss landscape more effectively.

There are however many other algorithms for updating the weights, such as Momentum, RMSProp, Adam, etc., each with its own advantages and disadvantages depending on the specific problem being addressed.

Observación. Optimizers are usually used, which are algorithms that adjust the learning rate and other hyperparameters during training to improve convergence and performance. Some popular optimizers include Adam, RMSProp, and Adagrad.

8.3. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a type of neural network designed to handle sequential data. They are particularly effective for tasks such as natural language processing, time series analysis, and speech recognition. They learn these temporal dependencies by maintaining a hidden state that is updated at each time step, allowing them to capture information from previous inputs in the sequence. The architecture of RNNs includes loops that allow information to persist, making them suitable for tasks where the order of the data is important. Three types of loops are commonly used in RNNs:

- Direct Connections: Connecting neurons to themselves, allowing them to retain information from previous time steps.
- Lateral Connections: Connecting neurons within the same layer, enabling them to share information across time steps.
- Indirect Connections: Connecting neurons to neurons in previous layers, allowing for more complex interactions between different time steps.

8.3.1. One-Directional RNNs

In one-directional RNNs, the information flows in one direction, from the input layer to the output layer. This means that the network can only capture dependencies from previous time steps, making it less effective for tasks that require understanding of future context. One-directional RNNs are commonly used for tasks such as language modeling and time series prediction, where the goal is to predict the next word in a sentence or the next value in a sequence based on the previous context.

8.3.2. Bidirectional RNNs

Bidirectional RNNs are designed to capture dependencies in both directions, allowing them to understand the context from both past and future time steps. This is achieved by having two separate hidden layers: one that processes the input sequence in the forward direction and another that processes it in the backward direction. The outputs from both layers are then combined to produce the final output.

8.3.3. Long Short-Term Memory (LSTM) Networks

Given the vanishing gradient problem, RNNs can struggle to learn long-term dependencies in sequences. To address this issue, Long Short-Term Memory (LSTM) networks were introduced. LSTMs are a type of RNN that includes special units called memory cells, which can maintain information for long periods of time. These cells are controlled by four gates:

- Input Gate: Controls how much of the new information should be stored in the memory cell.
- Forget Gate: Controls how much of the existing information in the memory cell should be retained or discarded.
- Output Gate: Controls how much of the information in the memory cell should be output to the next layer.
- Candidate Gate: Generates new candidate values that could be added to the memory cell.

The LSTM architecture allows it to learn long-term dependencies in sequences, making it effective for tasks such as language modeling, machine translation, and speech recognition. By controlling the flow of information through the gates, LSTMs can selectively retain or discard information, allowing them to capture complex patterns in sequential data.

Gated Recurrent Units (GRUs)

Gated Recurrent Units (GRUs) are a simplified version of LSTMs that also address the vanishing gradient problem. GRUs use only three gates:

- Update Gate: Controls how much of the previous information should be retained and how much of the new information should be added to the memory cell.
- Reset Gate: Controls how much of the previous information should be ignored when computing the new candidate values.
- Candidate Gate: Generates new candidate values that could be added to the memory cell.

Minimal Gated Recurrent Units (M-GRUs)

Minimal Gated Recurrent Units (M-GRUs) are an even simpler version of GRUs that use only two gates:

- Update Gate: Controls how much of the previous information should be retained and how much of the new information should be added to the memory cell.
- Candidate Gate: Generates new candidate values that could be added to the memory cell.

9. Advanced Neural Networks

9.1. Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a class of deep neural networks that have proven to be highly effective for tasks involving image and video recognition, as well as other applications that require spatial hierarchies of features. Before diving into the architecture of CNNs, it is essential to understand the fundamental components that make them unique.

9.1.1. Convolution

Convolution is the core operation in CNNs, which is used to extract features from input data. Humans can easily recognize objects in images by identifying patterns and features, such as edges, textures, and shapes. However, for a computer to recognize these features, it needs to learn how to detect them through a process called convolution. It consists of:

- The input image, which is typically represented as a multi-dimensional array (tensor).
- A filter (aka kernel), which is a smaller-sized matrix that is used to detect specific features in the input image.
- A *feature map*, which is the output of the convolution operation, representing the presence of specific features in the input image.

In order to obtain the feature map, the filter is slid over the input image, and at each position, an *element-wise* multiplication is performed between the filter and the corresponding region of the input image. The results are then summed up to produce a single value in the feature map. Some important parameters in the convolution operation include:

- Stride: The number of pixels by which the filter is moved across the input image. A larger stride results in a smaller feature map.
- Padding: This process has two main problems:
  - The size of the feature map is reduced after each convolution operation, which can lead to a loss of information, especially for deeper networks.
  - The edges of the input image are not treated equally, as the filter has fewer neighboring pixels to convolve with at the borders, which can lead to artifacts in the feature map.

In order to address these issues, padding is applied to the input image before the convolution operation. It involves adding extra pixels (usually zeros) around the borders of the input image, allowing the filter to be applied to the edges and preserving the spatial dimensions of the feature map. Some common padding techniques include:

- Constant Padding: Adding a constant value (usually zero) around the borders of the input image.
- Reflective Padding: Mirroring the values of the input image at the borders.
- Replicate Padding: Repeating the values of the input image at the borders.

Some specific filters are:

- Convolution Box Blur Filter: This filter is used to blur an image by averaging the pixel values in a local neighborhood.

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

- Convolution Gaussian Blur Filter: This filter is used to blur an image while preserving edges. It is based on the Gaussian function, which gives more weight to the central pixels in the local neighborhood.

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

- Convolution Edge Detection Filter: This filter is used to detect edges in an image by highlighting areas of rapid intensity change.

$$\begin{bmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{bmatrix}$$

9.1.2. Pooling

Pooling is a downsampling operation that reduces the spatial dimensions of the feature maps while retaining the most important information. It helps to reduce the computational complexity of the network and makes it more robust to variations in the input data, such as translations and distortions. There are several types of pooling operations, including:

- Max Pooling: This operation selects the maximum value from a local neighborhood of the feature map. It is the most commonly used pooling technique and helps to retain the most prominent features in the feature map.

- Average Pooling: This operation computes the average value of a local neighborhood of the feature map. It is less commonly used than max pooling, as it tends to blur the features and may not preserve the most important information.

- ROI Pooling: This operation is used in object detection tasks, where the input feature map is divided into a fixed number of Regions of Interest (ROIs), and *each ROI is pooled to a fixed size*. This is useful for handling objects of varying sizes in the input image, as it allows the network to work with a fixed-size representation of the ROIs, regardless of their original size.

9.1.3. Flattening

Flattening is the process of converting the multi-dimensional feature maps into a one-dimensional vector, which can then be fed into fully connected layers for classification or regression tasks. This step is essential for transitioning from the convolutional and pooling layers to the dense layers of the network.

9.1.4. Architecture of CNNs

A typical CNN architecture is divided into several layers, each serving a specific purpose. The main layers in a CNN include:

- Input Layer: This layer receives the input data, which is usually an image represented as a multi-dimensional array (tensor). The input layer does not perform any computations but serves as the entry point for the data into the network.

- Convolutional Layers: These layers perform the convolution operation, extracting features from the input data using *learnable* filters.

  Each convolutional layer is typically followed by an activation function, such as ReLU (Rectified Linear Unit), which introduces non-linearity into the network and allows it to learn complex patterns in the data.

- Pooling Layers: These layers perform the pooling operation, reducing the spatial dimensions of the feature maps and retaining the most important information.

- Flattening Layer: After the convolutional and pooling layers, the feature maps are flattened into a one-dimensional vector, which can then be fed into the fully connected layers for classification or regression tasks.

- Fully Connected Layers: Using the flattened feature vector, these layers perform the final classification or regression tasks. Each neuron in a fully connected layer is connected to every neuron in the previous layer, allowing the network to learn complex relationships between the features extracted by the convolutional and pooling layers.

- Output Layer: This layer produces the final output of the network, which can be a class label for classification tasks or a continuous value for regression tasks. The output layer typically uses an activation function, such as softmax for multi-class classification or sigmoid for binary classification.

9.2. Generative Adversarial Networks (GANs)

GANs are a class of deep learning models that consist of two neural networks, a generator and a discriminator, which are trained simultaneously in a competitive setting.

- The generator network takes random noise as input and generates synthetic data samples that resemble real images.
- The discriminator network takes both real and synthetic data samples as input and learns to distinguish between them, outputting a probability score indicating whether the input is real or fake.

9.3. Autoencoders

Autoencoders are a type of neural network used for unsupervised learning, where the goal is to reconstruct the input data as accurately as possible. They consist of:

- An encoder network that compresses the input data into a lower-dimensional representation, called the latent space.
- The latent space representation, which captures the most important features of the input data while discarding noise and irrelevant information.
- A decoder network that reconstructs the original input data from the latent representation.

The concept of the latent space has a great potential, as latent variables can be combined to generate new data samples.

10. Exercises

10.0. Introduction

The Python part of this exercise is available in this Jupyter Notebook.

Intelligence Tests

Ejercicio 1. Turing Test vs. Lovelace Test: Which test focuses on creativity and which on simulating conversation? How do these two tests work?

The Turing Test focuses on simulating conversation. It works by having a human evaluator engage in a natural language conversation with both a machine and a human without knowing which is which. If the evaluator cannot reliably distinguish the machine from the human, the machine is considered to have passed the Turing Test.

The Lovelace Test, on the other hand, focuses on creativity. It requires an AI to create something original (like a piece of art, music, or a story) that cannot be attributed to any human creator. The AI must be able to explain how it created the work and why it is original, demonstrating a level of creativity that goes beyond mere imitation.

Ejercicio 2. Why do many simple chatbots fail the Winograd Schema?

Many simple chatbots fail the Winograd Schema because they lack the necessary contextual understanding and common sense reasoning required to correctly interpret the ambiguous sentences used in the test. The Winograd Schema consists of sentences that have a pronoun with an ambiguous antecedent, and the correct interpretation depends on understanding the context and the relationships between the entities involved. Simple chatbots often rely on pattern matching or statistical methods rather than true comprehension, which leads to incorrect answers when faced with such nuanced language challenges.

Ejercicio 3. What is the main idea behind the Metzinger test for checking if an AI is self-aware, in simple terms? What is one thing that might make this test difficult to use for all types of AI?

The main idea behind the Metzinger test is to check if an AI can understand and describe its own internal processes and experiences, which would indicate self-awareness. One difficulty with this test is that it relies on the AI's ability to communicate its thoughts and feelings, which may not be possible for all types of AI,

especially those that do not have a natural language processing capability or a way to express their internal states effectively. Therefore, there is no universally applicable method to determine if an AI is self-aware, as it may depend on the specific design and capabilities of the AI in question.

10.1. Data Processing / Cleaning

The Python part of this exercise is available in this Jupyter Notebook.

Data Filtering and Transforming

Ejercicio 10.1.1. Imagine you have a list of all the students in a school. You only want to look at the students in the 5th grade (data filtering). Then, you want to calculate the average age of those 5th graders (data transforming). Explain why you need both steps.

We first need to filter the data to focus only on the students in the 5th grade, as we are specifically interested in that group. This step allows us to narrow down our dataset to the relevant subset of students. After filtering, we need to transform the data by calculating the average age of those 5th graders. This transformation step is necessary to derive meaningful insights from the filtered data, as it provides us with a summary statistic (the average age) that can help us understand the characteristics of the 5th-grade students. Without filtering, we would be calculating the average age of all students, which would not give us the specific information we are looking for about the 5th graders.

Ejercicio 10.1.2. You have a list of toys that were sold in a store, including the toy's name and the price it sold for.

1. Give an example of a question you could answer by only looking at toys that cost more than 20 (data filtering).

   A question that could be answered by data filtering would be: "Which toys that cost more than 20 euros were sold in the last month?" (Here, we filter the data by price greater than 20 euros and potentially by the sales date).

2. Give an example of something you could calculate or change about the toy prices (data transforming).

   An example of data transformation would be calculating the average selling price of all toys or converting all prices from euros to another currency (e.g., US dollars) based on a specific exchange rate. Another transformation could be creating a new column indicating whether a toy is classified as "cheap" (e.g., < 15 euros), "mid-priced" (e.g., 15 euros $-$ 30 euros), or "expensive" ($>$ 30 euros) based on the selling price.

Data Understanding

Ejercicio 10.1.3. What is the difference between quantitative (numerical) and qualitative (categorical) data?

Quantitative data refers to numerical values that can be measured and quantified, such as height, weight, or age. Qualitative data, on the other hand, refers to categorical values that describe characteristics or attributes, such as color or gender. Quantitative data can be analyzed using mathematical operations, while qualitative data is typically analyzed using descriptive statistics or categorization.

Ejercicio 10.1.4. What does the abbreviation `NaN` mean in a dataset?

`NaN` stands for "Not a Number" and is used in datasets to represent missing or undefined values. It is also used to indicate that a value is not a valid number, such as the result of an undefined mathematical operation (e.g., division by zero). In data analysis, `NaN` values need to be handled appropriately, either by filling them with a specific value or dropping them.

Ejercicio 10.1.5. Briefly explain the difference between the mean and the median. Which value is more sensitive to outliers?

The mean is the average of a set of numbers, calculated by summing all the values and dividing by the count of values. The median is the middle value in a sorted list of numbers. The mean is more sensitive to outliers because it can be significantly affected by extremely high or low values, while the median is less affected as it only considers the middle value(s) and not the magnitude of all values.

Ejercicio 10.1.6. Calculate the quartiles for the following dataset: [10, 9, 1, 4, 3, 2, 5, 6, 6, 6, 3, 9].

First, we need to sort the dataset:

$$[1, 2, 3, 3, 4, 5, 6, 6, 6, 9, 9, 10]$$

The quartiles are then:

$$Q2 = \frac{6+5}{2} = 5{,}5$$
$$Q1 = \frac{3+3}{2} = 3$$
$$Q3 = \frac{6+9}{2} = 7{,}5$$

Data Cleaning

Ejercicio 10.1.7. Name two reasons why data is often dirty in the real world.

Data can be dirty due to:

- Human error during data entry, which can lead to typos, incorrect values, or missing data.
- Inconsistent data formats or structures, such as different date formats or varying units of measurement across datasets.

Ejercicio 10.1.8. What happens when you apply the `dropna()` function to a DataFrame?

The `dropna()` function removes any rows (or columns, if specified with the `axis=1` parameter) that contain missing values (`NaN`) from the DataFrame. The original DataFrame is not modified unless the `inplace=True` parameter is used.

This is a common data cleaning step to ensure that the dataset does not contain incomplete records that could affect analysis or modeling.

Ejercicio 10.1.9. Why do you scale (normalize) numerical data before feeding it into a machine learning model?

Scaling (normalizing) numerical data is important because many machine learning algorithms are sensitive to the scale of the input features. If the features have different ranges, the algorithm may give more weight to those with larger values, leading to biased results. Scaling helps to ensure that all features contribute equally to the model's learning process and can improve the convergence speed and performance of the model.

Ejercicio 10.1.10. Name two possible encodings and explain them using an example.

They are explained in the page 13.

Ejercicio 10.1.11. Describe three common challenges encountered during data preparation and explain a technique to mitigate each challenge.

- Missing Data: This can be mitigated by using techniques such as imputation (filling in missing values with the mean, median, or mode) or by dropping rows/columns with a high percentage of missing values.
- Outliers: Outliers can skew the results of analysis and modeling. They can be mitigated by using methods such IQR to identify and either remove or transform outliers.
- Inconsistent Data: Inconsistent data can arise from different formats or units. This can be mitigated by standardizing the data format (e.g., converting all dates to a common format) and ensuring that all measurements are in the same units (e.g., converting all weights to kilograms).

Ejercicio 10.1.12. Explain the difference between data cleaning and data transformation. Provide an example of each.

Data Cleaning Data cleaning involves identifying and correcting errors or inconsistencies in the dataset to ensure that it is accurate and reliable. For example, removing duplicate entries from a customer database or filling in missing values in a sales dataset.

Data Transformation Data transformation involves converting data from one format or structure to another to make it suitable for analysis or modeling. For example, normalizing numerical features to a common scale or encoding categorical variables into numerical format using one-hot encoding.

Ejercicio 10.1.13. Why is it important to handle missing data appropriately? Discuss three different strategies for dealing with missing values and their potential drawbacks.

Handling missing data appropriately is crucial because it can significantly impact the results of analysis and modeling. If not addressed, missing data can lead to biased estimates, reduced statistical power, and inaccurate predictions. Three strategies for dealing with missing values are:

- Imputation: Filling in missing values with a specific value (e.g., mean, median, mode) can help maintain the dataset's size and structure. However, it can introduce bias if the imputed values do not accurately reflect the underlying distribution of the data.
- Deletion: Removing rows or columns with missing values can be simple but may lead to loss of information and reduced sample size.
- Model-based Imputation: Using statistical models to predict and fill in missing values based on other available data. This approach can be more accurate but is computationally intensive and may introduce additional complexity in model interpretation.

Ejercicio 10.1.14. What are outliers in a dataset and why can they be problematic? Describe three methods for detecting outliers.

Outliers are data points that significantly differ from the majority of the data. They can be problematic because they can skew the results of analysis and modeling, leading to inaccurate conclusions. Three methods for detecting outliers are:

- Z-score: This method identifies outliers by calculating the number of standard deviations a data point is from the mean. A common threshold is a Z-score of 3 or -3.
- Interquartile Range (IQR): This method identifies outliers by calculating the IQR (the difference between the third quartile and the first quartile) and defining outliers as data points that fall below $Q1 - 1{,}5 \cdot IQR$ or above $Q3 + 1{,}5 \cdot IQR$.
- Visual Methods: Box plots and scatter plots can visually reveal outliers in the data, allowing for easy identification of data points that deviate from the overall pattern.

Ejercicio 10.1.15. Explain the concept of data normalization or standardization. When would you prefer one over the other, and why is this a crucial step in many machine learning workflows?

Data normalization (scaling to a range) and standardization (scaling to have a mean of 0 and a standard deviation of 1) are techniques used to rescale numerical features. Normalization is preferred when the data does not follow a Gaussian distribution and when the algorithm does not assume any distribution. Standardization is preferred when the data follows a Gaussian distribution or when the algorithm assumes that the data is normally distributed. This step is crucial in machine learning workflows because it ensures that all features contribute equally to the model's learning process and can improve the convergence speed and performance of the model.

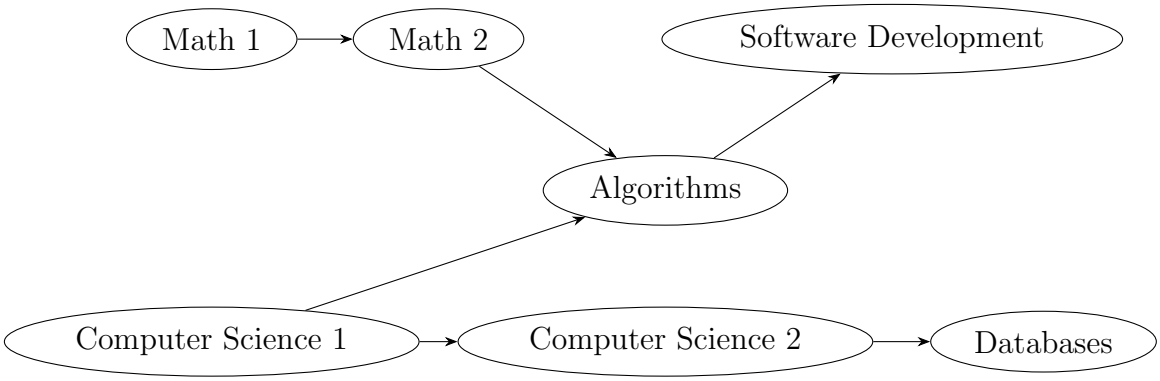

Figura 10.1: Directed graph representing the dependencies between lectures.

10.2. Search Algorithms

The Python part of this exercise is available in this Jupyter Notebook.

Kahn's Algorithm (Topological Sorting)

You are responsible for the course planning at a university. For a specific module, there are several lectures that partially build upon each other. Your task is to find an order in which all lectures can be attended without attending a lecture before one of its prerequisites.

Lectures (Nodes): Math 1, Math 2, Computer Science 1, Computer Science 2, Databases, Algorithms, Software Development

Dependencies (directed edges "must be attended before"):

- Math 1 → Math 2
- Computer Science 1 → Computer Science 2
- Computer Science 1 → Algorithms
- Math 2 → Algorithms
- Computer Science 2 → Databases
- Algorithms → Software Development

Ejercicio 10.2.1. Create the graph. Represent the directed graph as an adjacency list, and for each node, note the in-degree (number of incoming edges).

The graph is depicted in Figure 10.1. The adjacency list and in-degrees are as follows:

- Math 1: Adjacent to [Math 2], In-degree = 0
- Math 2: Adjacent to [Algorithms], In-degree = 1
- Computer Science 1: Adjacent to [CS 2, Algorithms], In-degree = 0
- Computer Science 2: Adjacent to [Databases], In-degree = 1

- Databases: Adjacent to [], In-degree = 1
- Algorithms: Adjacent to [Software Development], In-degree = 2
- Software Development: Adjacent to [], In-degree = 1

Ejercicio 10.2.2. Apply Kahn's Algorithm. Perform Kahn's algorithm manually (or implement it in a programming language of your choice) to find a topological sorting of the lectures. Document each step:

1. Start with all nodes whose in-degree = 0.
2. Remove one of these nodes, add it to the order, and reduce the in-degrees of its neighbors.
3. Repeat until all nodes are processed.

The algorithm can be seen in the Table 10.1.

Ejercicio 10.2.3. Interpret the result. State the calculated order of the lectures, and determine if this order is unique. Justify your answer.

The calculated order of the lectures is:

Math 1 → Math 2 → Computer Science 1 → Computer Science 2 → Databases → Algorithms → Software Development

This order is not unique. For example, Computer Science 1 and Math 1 both have an in-degree of 0 at the start, so we could have chosen Computer Science 1 before Math 1, which would lead to a different valid topological order. The presence of multiple nodes with in-degree 0 at various steps allows for multiple valid topological sorts.

BFS vs DFS

Ejercicio 10.2.4. What are the key differences between BFS and DFS in terms of traversal strategy, data structures used, and typical applications?

Traversal Strategy: BFS explores all neighbors at the present depth before moving on to the nodes at the next depth level, while DFS explores as far as possible along each branch before backtracking.

Data Structures: BFS typically uses a queue to keep track of the next nodes to visit, while DFS uses a stack (either explicitly or via recursion).

Applications: BFS is often used for finding the shortest path in unweighted graphs, while DFS is useful for tasks like topological sorting, cycle detection, and solving puzzles that require exploring all possibilities (e.g., mazes).

Ejercicio 10.2.5. In what scenarios would you prefer BFS over DFS, and vice versa?

Tabla 10.1: Steps of Kahn's Algorithm for topological sorting of the lectures.

Step	**Final Order**	**New In-degrees**
0	[]	Math 1: 0 Math 2: 1 Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 2 Software Development: 1
1	[Math 1]	Math 2: 0 Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 2 Software Development: 1
2	[Math 1, Math 2]	Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 1 Software Development: 1
3	[Math 1, Math 2, Computer Science 1]	Computer Science 2: 0 Databases: 1 Algorithms: 0 Software Development: 1
4	[M1, M2, CS1, CS2]	Databases: 0 Algorithms: 0 Software Development: 1
5	[M1, M2, CS1, CS2, Databases]	Algorithms: 0 Software Development: 1
6	[M1, M2, CS1, CS2, Databases, Algorithms]	Software Development: 0
7	[M1, M2, CS1, CS2, Databases, Algorithms, SD]	

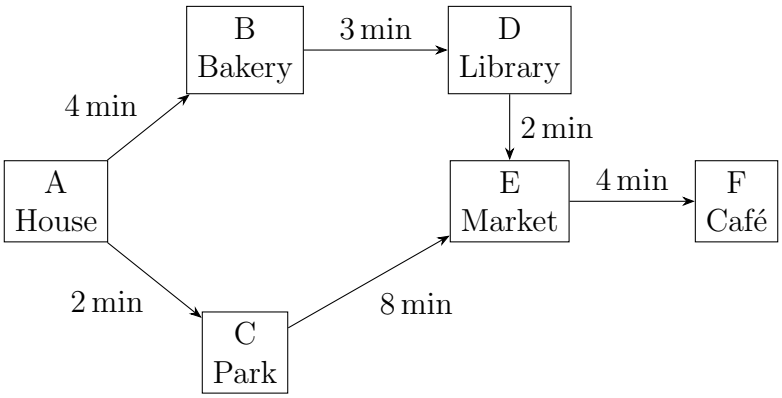

Figura 10.2: Graph for Exercise 10.2.6.

Prefer BFS: When you need to find the shortest path in an unweighted graph, when you want to explore all nodes at a certain depth before going deeper, or when the solution is likely to be found close to the root.

Prefer DFS: When you want to explore all possible paths (e.g., backtracking problems), when the solution is likely to be found far from the root, or when you want to use less memory (DFS can be more memory-efficient than BFS in certain cases).

Ejercicio 10.2.6. Create a small graph as an example to practice this task (feel free to include a few intersections and cafés).

The graph is depicted in Figure 10.2.

Ejercicio 10.2.7 (Approach)**.** Explain how you would find the café if you first searched all directly adjacent intersections (BFS), compared to simply walking straight down a street until you can go no further (DFS).

BFS Approach: Starting at the house (A), you would first check all the adjacent intersections (B and C). If you find the café at either of these intersections, you are done. If not, you would then check the neighbors of B (D) and C (E). If you find the café at D or E, you are done. If not, you would check the neighbors of D (E, which you wouldn't visit as you have already visited it) and E (F), where you would find the café at F.

DFS Approach: Starting at the house (A), you would choose one adjacent intersection to explore first (e.g., B). From B, you would go to D, and from D to E, and finally from E to F, where you would find the café. You would not check C at all in this approach, and you would only find the café after exploring a path with more intersections.

Ejercicio 10.2.8. Why does a queue help you search systematically in a circle around your starting point, while a stack tends to push you deeper into the structure?

A queue helps you search systematically in a circle around your starting point because it processes nodes in the order they were added. When you add all adjacent

nodes to the queue, you will explore all nodes at the current depth before moving on to the next level. This ensures that you are exploring nodes in a breadth-first manner, which naturally expands outwards from the starting point.

In contrast, a stack processes nodes in a last-in, first-out (LIFO) order. When you add an adjacent node to the stack, it will be the next node you explore. This means that you will go deeper into the structure along one path before backtracking to explore other paths. This depth-first approach can lead you to explore a long path before checking other adjacent nodes, which is why it tends to push you deeper into the structure.

Uniform Cost Search (UCS)

Ejercicio 10.2.9. Explain the concept of Uniform Cost Search. How does it differ from BFS and DFS?

Uniform Cost Search (UCS) is a search algorithm that expands the node with the lowest cumulative cost from the start node. It differs from BFS and DFS in that it takes into account the cost of reaching each node, rather than just the depth (BFS) or the path taken (DFS). UCS uses a priority queue to manage the nodes based on their cumulative cost, ensuring that it always expands the least costly path first. This makes UCS suitable for finding the optimal solution in weighted graphs, whereas BFS and DFS do not guarantee optimality in such cases.

Ejercicio 10.2.10. What data structure does Uniform Cost Search typically use to manage the search process?

Uniform Cost Search typically uses a priority queue to manage the search process. The priority queue allows UCS to always expand the node with the lowest cumulative cost first, ensuring that it finds the optimal solution in weighted graphs.

Ejercicio 10.2.11. When is Uniform Cost Search most appropriate?

Uniform Cost Search is most appropriate when the graph has weighted edges and you need to find the path with the minimum total cost from the start node to a goal node.

Ejercicio 10.2.12 (Approach)**.** Suppose each street takes a different amount of time (traffic lights, inclines). In UCS, how do you decide which turn to take next?

In Uniform Cost Search, you would calculate the cumulative cost (time) to reach each adjacent intersection from your current location. You would then add these costs to a priority queue, which will allow you to always expand the node with the lowest cumulative cost next. This way, you ensure that you are always taking the path that has cost you the least effort so far, leading you towards the optimal route to your destination.

Ejercicio 10.2.13. How do you ensure that you always continue along the path that has cost you the least effort so far?

To ensure that you always continue along the path that has cost you the least effort so far, you would use a priority queue to manage the nodes based on their cumulative cost. Whenever you expand a node, you would calculate the cumulative cost to reach its neighbors and add them to the priority queue. The priority queue will automatically sort the nodes based on their cumulative cost, allowing you to

always expand the node with the lowest cost next. This way, you are guaranteed to follow the path that has cost you the least effort at each step, leading you towards the optimal solution.

Dijkstra's Algorithm

Ejercicio 10.2.14. Describe Dijkstra's algorithm. What problem does it solve?

Dijkstra's algorithm is a graph search algorithm that finds the shortest path from a starting node to all other nodes in a weighted graph. It solves the problem of finding the minimum cumulative cost (or distance) from the start node to each of the other nodes in the graph. The algorithm uses a priority queue to keep track of the nodes to be explored, always expanding the node with the lowest cumulative cost first. This ensures that once a node's shortest path is determined, it will not change, allowing Dijkstra's algorithm to efficiently find the optimal paths in weighted graphs.

Ejercicio 10.2.15. How does Dijkstra's algorithm handle weighted graphs?

Dijkstra's algorithm handles weighted graphs by maintaining a priority queue that keeps track of the cumulative cost to reach each node from the starting node. When the algorithm expands a node, it calculates the cumulative cost to reach its neighbors by adding the weight of the edge connecting them to the cumulative cost of the current node. If this new cumulative cost is less than the previously recorded cost for that neighbor, the algorithm updates the cost and adds the neighbor to the priority queue for further exploration. This process ensures that Dijkstra's algorithm efficiently finds the shortest paths in weighted graphs.

Ejercicio 10.2.16. Compare and contrast Dijkstra's algorithm with Uniform Cost Search.

Dijkstra's algorithm and Uniform Cost Search (UCS) are the same algorithm, but UCS stops when it reaches the goal node, while Dijkstra's algorithm continues until it has found the shortest path to all nodes in the graph.

Ejercicio 10.2.17 (Procedure)**.** Describe the step-by-step process Dijkstra uses to determine the distance from the starting point to each intersection.

Dijkstra's algorithm starts by initializing the distance to the starting point as 0 and the distances to all other nodes as infinity. It uses a priority queue to keep track of the nodes to explore, starting with the initial node. At each step, it expands the node with the lowest cumulative cost (distance) from the start node. For each of its neighbors, it calculates the cumulative cost to reach that neighbor by adding the weight of the edge connecting them to the cumulative cost of the current node. If this new cumulative cost is less than the previously recorded cost for that neighbor, it updates the cost and adds the neighbor to the priority queue for further exploration. This process continues until all nodes have been explored, resulting in the shortest distance from the starting point to each intersection.

Ejercicio 10.2.18. What do you do if you discover a shortcut to an intersection that you previously knew only via a longer route?

If you discover a shortcut to an intersection that you previously knew only via a longer route, you would update the cumulative cost (distance) for that intersection

in the priority queue. This involves checking if the new cumulative cost is less than the previously recorded cost for that intersection. If it is, you would update the cost and add the intersection back into the priority queue for further exploration. This way, Dijkstra's algorithm ensures that it always finds the shortest path to each node, even if a shorter path is discovered after initially exploring a longer route.

A* Search

Ejercicio 10.2.19. Explain the A* search algorithm. What is the role of the heuristic function in A*?

A* search algorithm is an informed search algorithm that combines the features of Uniform Cost Search and a heuristic to efficiently find the shortest path from a start node to a goal node in a weighted graph. The role of the heuristic function in A* is to estimate the cost from the current node to the goal node. This heuristic helps guide the search process by prioritizing nodes that are likely to lead to the goal more quickly, thus reducing the number of nodes explored and improving efficiency compared to uninformed search algorithms like Uniform Cost Search.

Ejercicio 10.2.20. How does A* differ from Dijkstra's algorithm and Uniform Cost Search?

A* differs from Dijkstra's algorithm and Uniform Cost Search in that it incorporates a heuristic function to estimate the cost from the current node to the goal node. While Dijkstra's algorithm and Uniform Cost Search only consider the cumulative cost from the start node to the current node, A* combines this cumulative cost with the heuristic estimate to prioritize nodes that are more likely to lead to the goal. This allows A* to find the optimal path more efficiently in many cases, especially when the heuristic is well-designed.

Ejercicio 10.2.21. What are the conditions for a heuristic to be admissible and consistent in A*?

- Admissible: A heuristic is admissible if it never overestimates the true cost to reach the goal. In other words, for any node n, the heuristic value $h(n)$ must be less than or equal to the actual cost from n to the goal node.
- Consistent (Monotonic): A heuristic is consistent if for every node n and every successor n' of n, the estimated cost to reach the goal from n is less than or equal to the cost of reaching n' plus the estimated cost from n' to the goal, which is basically the triangle inequality. Consistency implies admissibility, but not vice versa.

Ejercicio 10.2.22 (Procedure)**.** You see the church tower in the distance near the café. How does A* combine the distance already traveled with this straight-line estimate to shorten the route?

A* combines the distance already traveled (the cumulative cost from the start node to the current node) with the straight-line estimate (the heuristic value from the current node to the goal node) by calculating a total cost function $f(n) = g(n)+h(n)$, where $g(n)$ is the cumulative cost to reach node n and $h(n)$ is the heuristic estimate to reach the goal from node n. The algorithm prioritizes nodes with the lowest $f(n)$

value, which means it will explore paths that are both cost-effective so far and have a promising estimate to reach the goal. This allows A* to efficiently find a shorter route to the café by guiding the search towards nodes that are likely to lead to the goal more quickly.

Ejercicio 10.2.23. Why can't your guess (I think it's still 5 minutes) be longer than the actual distance, so that you're guaranteed to find the café via the fastest route?

Your guess (the heuristic estimate) cannot be longer than the actual distance because if it were, it would overestimate the cost to reach the goal. This would lead A* to prioritize nodes that are actually farther from the goal, potentially causing it to explore less optimal paths and miss the fastest route. By ensuring that the heuristic is admissible (never overestimates), A* guarantees that it will find the optimal path to the café, as it will always consider paths that are truly closer to the goal as more promising.

Tabla 10.2: Confusion matrix for the spam filter model.

	Actual: Spam	**Actual: Not Spam**
Model says: Spam	30 (TP)	10 (FP)
Model says: Not Spam	20 (FN)	40 (TN)

10.3. Machine Learning Algorithms

The Python part of this exercise is available in this Jupyter Notebook.

The Intelligent Spam Filter

Imagine you have trained a model to distinguish important emails from annoying advertisements. To evaluate the quality of your filter, you ran a test on 100 emails and obtained the results shown in Table 10.2.

Confusion Matrix

Ejercicio 10.3.1. State the formulas for Accuracy, Precision, Recall, and F1-Score using TP, FP, TN, FN, and specify their respective ranges.

- Accuracy: It is ranged in $[0, 1]$.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- Precision: It is ranged in $[0, 1]$.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall: It is ranged in $[0, 1]$.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- F1-Score: It is ranged in $[0, 1]$.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ejercicio 10.3.2. Calculate the values using the table above. What do these values specifically say about the reliability of your filter in everyday use?

- Accuracy: $\dfrac{30 + 40}{30 + 10 + 40 + 20} = \dfrac{70}{100} = 0{,}7.$
- Precision: $\dfrac{30}{30 + 10} = \dfrac{30}{40} = 0{,}75.$
- Recall: $\dfrac{30}{30 + 20} = \dfrac{30}{50} = 0{,}6.$

- F1-Score: $2 \cdot \dfrac{0{,}75 \cdot 0{,}6}{0{,}75 + 0{,}6} = 2 \cdot \dfrac{0{,}45}{1{,}35} = 2 \cdot \dfrac{1}{3} = \dfrac{2}{3} \approx 0{,}67.$

These values indicate that while the filter is reasonably accurate (70 %), it has a moderate precision (75 %) and a lower recall (60 %). This means that while most of the emails it identifies as spam are indeed spam, it misses a significant portion (40 %) of actual spam emails, which could lead to important emails being overlooked. In addition, the F1-Score of approximately 0.67 suggests that there is room for improvement in balancing precision and recall for better overall performance in everyday use.

Ejercicio 10.3.3. To get a complete picture, we will also use the Fowlkes-Mallows Index (FMI). What is the formula, and calculate the value.

$$\text{FMI} = \sqrt{\text{Precision} \cdot \text{Recall}} = \sqrt{0{,}75 \cdot 0{,}6} \approx 0{,}67$$

Ejercicio 10.3.4. Which value is more sensitive to the missed spam emails (FN) in your example?

The Recall is more sensitive to the missed spam emails (FN) because it directly measures the proportion of actual spam emails that were correctly identified by the model. A high number of false negatives (missed spam emails) will significantly decrease the Recall, indicating that the model is not effectively capturing all the spam emails in the dataset.

Ejercicio 10.3.5. Suppose your filter has an accuracy of 100 % on the training data, but only 70 % in the test table above. What phenomenon is at play here? Briefly explain why memorized emails hinder generalization.

The phenomenon at play here is called *overfitting*. Overfitting occurs when a model learns the training data too well, including its noise and outliers, to the point that it performs exceptionally on the training data but poorly on unseen test data. Memorized emails hinder generalization because the model has essentially "memorized" specific examples from the training set rather than learning the underlying patterns that can be applied to new, unseen data. As a result, when the model encounters new emails in the test set, it fails to generalize and correctly classify them, leading to a significant drop in accuracy.

Ejercicio 10.3.6. Explain the variance-bias dilemma: Which of these two factors predominates when the model begins to learn the noise in the data (overfitting), and which predominates when the model is oversimplified (underfitting)?

The variance-bias dilemma refers to the trade-off between two sources of error in machine learning models: bias and variance.

- Bias: Bias is the error introduced by approximating a real-world problem, which may be complex, by a simplified model. High bias can lead to underfitting, where the model is too simple to capture the underlying patterns in the data.

- Variance: Variance is the error introduced by the model's sensitivity to small fluctuations in the training data. High variance can lead to overfitting, where the model learns not only the underlying patterns but also the noise in the training data.

Ejercicio 10.3.7. How does the complexity of the model change when you try to reduce the bias?

When you try to reduce the bias, you typically increase the complexity of the model. This is because reducing bias often involves adding more parameters or using a more flexible model that can capture more complex patterns in the data. However, increasing the complexity of the model can also lead to higher variance, which may result in overfitting if not managed properly.

Data Imbalance (Resampling)

Ejercicio 10.3.8. The needle-in-a-haystack problem: Your dataset contains 9900 normal emails and only 100 spam emails. Explain why a model without resampling tends to simply always predict Not Spam. What would the recall value be in this extreme case?

In this extreme case, a model without resampling would likely learn to predict Not Spam for all emails because it would achieve a high accuracy of 99 % by doing so (9900 out of 10000 emails are Not Spam). However, this approach completely ignores the minority class (Spam), leading to a recall value of 0 % for the Spam class. This is because the model would fail to identify any of the actual spam emails, resulting in zero true positives (TP) and thus a recall of:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{0}{0 + 100} = 0$$

Resampling techniques are crucial to address this imbalance and allow the model to learn to identify spam emails effectively. Some options include oversampling the minority class or undersampling the majority class to create a more balanced dataset for training.

Ejercicio 10.3.9. Why is it crucial to apply resampling to the training data and not to the test data? What would happen to the validity of your metrics if the test table were artificially balanced?

It is crucial to apply resampling only to the training data because the test data should reflect the real-world distribution of classes to provide an accurate evaluation of the model's performance. If the test table were artificially balanced, it would not represent the actual conditions under which the model will operate, leading to misleading metrics. For instance, if the test set is balanced, the model might appear to perform well in terms of recall and precision for both classes, but in reality, it may struggle to identify spam emails in a real-world scenario where they are much less frequent. This would compromise the validity of the metrics and give a false sense of confidence in the model's performance.

Automatic Fruit Sorting (kNN)

You are using a sensor that classifies fruit based on two features: weight (x_1) and hardness (x_2).

The k-Nearest-Neighbors Algorithm

Ejercicio 10.3.10. An unknown piece of fruit lands on the conveyor belt. Briefly describe the three algorithmic steps that kNN now performs to determine whether it is an apple or a pear.

- Step 1: Calculate Distances: The algorithm calculates the distance between the unknown fruit and all the known fruits in the training dataset using a distance metric.
- Step 2: Identify Neighbors: The algorithm identifies the k nearest neighbors (the k closest known fruits) based on the calculated distances.
- Step 3: Make a Prediction: The algorithm makes a prediction for the unknown fruit by taking a majority vote among the k nearest neighbors. The class (apple or pear) that appears most frequently among the neighbors is assigned to the unknown fruit.

Ejercicio 10.3.11. What are the formulas for the Euclidean distance and the Manhattan distance between two points $P(x_1, x_2)$ and $Q(y_1, y_2)$?

The formulas for the distances are as follows:

- Euclidean Distance:

$$d_{\text{Euclidean}}(P, Q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

- Manhattan Distance:

$$d_{\text{Manhattan}}(P, Q) = |x_1 - y_1| + |x_2 - y_2|$$

Ejercicio 10.3.12. The new fruit has the coordinates $(5, 2)$. A known reference apple is located at $(10, 5)$. Calculate the distance using both of the above-mentioned distance methods.

- Euclidean Distance:

$$d_{\text{Euclidean}}((5,2),(10,5)) = \sqrt{(5-10)^2 + (2-5)^2} = \sqrt{25+9} = \sqrt{34} \approx 5{,}83$$

- Manhattan Distance:

$$d_{\text{Manhattan}}((5,2),(10,5)) = |5-10| + |2-5| = 5 + 3 = 8$$

k-Nearest Neighbors

Ejercicio 10.3.13. What are the fundamental differences in the operating principles of kNN, k-Means, and k-Medians, and what types of tasks are they typically used for (classification, regression, clustering)?

kNN (k-Nearest Neighbors): kNN is a supervised learning algorithm used for classification and regression tasks. It classifies a new data point based on the majority class among its k nearest neighbors in the feature space. For regression, it predicts the value based on the average of the target values of the k nearest neighbors.

k-Means: k-Means is an unsupervised learning algorithm used for clustering tasks. It partitions the data into k clusters by assigning each data point to the cluster with the nearest mean (centroid). The algorithm iteratively updates the centroids until convergence. k-Means is sensitive to outliers because the mean can be significantly affected by extreme values.

k-Medians: k-Medians is also an unsupervised learning algorithm used for clustering tasks, similar to k-Means. However, instead of using the mean to represent each cluster, it uses the median, which makes it more robust to outliers compared to k-Means.

Ejercicio 10.3.14. How exactly does kNN work step by step?

1. Distance Calculation: For a given new data point, the algorithm calculates the distance between this point and all the points in the training dataset using a chosen distance metric (e.g., Euclidean, Manhattan).
2. Neighbor Identification: The algorithm identifies the k nearest neighbors to the new data point based on the calculated distances.
3. Prediction: The algorithm makes a prediction for the new data point depending on the task:
   - For classification tasks, it assigns the class that is most common among the k nearest neighbors.
   - For regression tasks, it calculates the average (or median) of the target values of the k nearest neighbors and assigns this value to the new data point.

Ejercicio 10.3.15. What distance metrics do you know, and provide their mathematical forms?

- Euclidean Distance:

$$d_{\text{Euclidean}}(P,Q) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- Manhattan Distance:

$$d_{\text{Manhattan}}(P,Q) = \sum_{i=1}^{n}|x_i - y_i|$$

- Minkowski Distance:

$$d_{\text{Minkowski}}(P,Q) = \sqrt[p]{\sum_{i=1}^{n}|x_i - y_i|^p}$$

  where p is a parameter that can be adjusted to obtain different distance metrics (e.g., $p = 1$ for Manhattan, $p = 2$ for Euclidean).

Ejercicio 10.3.16. What are the advantages and disadvantages of kNN, k-Means, and k-Medians in terms of robustness to outliers, computational effort, and interpretability of results?

kNN:

- Advantages: Simple to understand and implement, can capture complex relationships in the data, and can be used for both classification and regression tasks.
- Disadvantages: Computationally expensive for large datasets, sensitive to irrelevant features and the choice of k, and does not provide a model or insights into feature importance.

k-Means:

- Advantages: Efficient for large datasets, easy to implement, and provides clear cluster assignments.
- Disadvantages: Sensitive to outliers, requires the number of clusters (k) to be specified in advance, and may converge to local minima.

k-Medians:

- Advantages: More robust to outliers than k-Means, as it uses the median instead of the mean to represent clusters.
- Disadvantages: Computationally more expensive than k-Means due to the need to calculate medians, and also requires the number of clusters (k) to be specified in advance.

Ejercicio 10.3.17. How do you choose the optimal value for "k" in kNN, k-Means, and k-Medians, and what methods are there to evaluate the performance of the models?

- kNN: The optimal value of k can be chosen using techniques such as cross-validation, where the dataset is split into training and validation sets to evaluate the performance of the model for different values of k. The value of k that yields the best performance (e.g., highest accuracy, lowest error) on the validation set is selected.
- k-Means and k-Medians: The optimal number of clusters (k) can be determined using methods such as the Elbow Method, which involves plotting the within-cluster sum of squares (WCSS) against different values of k and looking for an "elbow" point where the rate of decrease sharply changes.

Ejercicio 10.3.18. In what scenarios would k-Medians be preferable to k-Means, and what specific challenges arise in the implementation and application of k-Medians?

k-Medians would be preferable to k-Means in scenarios where the dataset contains outliers or is not normally distributed, as k-Medians is more robust to outliers due to its use of the median instead of the mean. However, challenges in implementing k-Medians include increased computational complexity compared to k-Means,

as calculating the median can be more time-consuming than calculating the mean (since it may require sorting the data). Additionally, like k-Means, k-Medians also requires the number of clusters (k) to be specified in advance, which can be difficult to determine without prior knowledge of the data.

10.4. Genetic Algorithms

The Python part of this exercise is available in this Jupyter Notebook.

Imagine a delivery driver who needs to visit a set of cities and return to the starting city. The goal is to find the shortest possible route that visits each city exactly once. This is a classic optimization problem known as the Traveling Salesperson Problem (TSP).

Genetic Algorithm Concepts

Ejercicio 10.4.1. Explain the fundamental concepts of genetic algorithms, including population, fitness function, selection, crossover, and mutation.

Genetic algorithms are a type of optimization algorithm inspired by the process of natural selection. They work by maintaining a population of candidate solutions, which evolve over time through selection, crossover, and mutation.

- Population: A set of candidate solutions to the optimization problem. Each solution is often represented as a chromosome (e.g., a string of bits, a permutation, etc.).
- Fitness Function: A function that evaluates the quality of each candidate solution. It assigns a fitness score based on how well the solution solves the problem.
- Selection: The process of choosing candidate solutions from the population to create offspring for the next generation. Common selection methods include roulette wheel selection, tournament selection, and rank-based selection.
- Crossover: A genetic operator that combines two parent solutions to produce one or more offspring. The crossover operator is designed to exchange information between the parents, potentially creating better solutions.
- Mutation: A genetic operator that introduces random changes to a candidate solution. Mutation helps maintain genetic diversity in the population and allows the algorithm to explore new areas of the solution space.

Ejercicio 10.4.2. How do genetic algorithms differ from traditional optimization algorithms?

While traditional optimization algorithms (like gradient descent, linear programming, etc.) often work with a single solution and improve it iteratively, genetic algorithms maintain a population of solutions and use stochastic processes to explore the solution space.

Ejercicio 10.4.3. What are the advantages and disadvantages of using genetic algorithms for optimization problems?

- Advantages:
  - Can handle complex, non-linear, and multi-modal optimization problems.

  - Does not require gradient information, making it suitable for problems where the fitness function is not differentiable.
  - Can escape local optima due to its stochastic nature.

- Disadvantages:
  - Can be computationally expensive, especially for large populations and many generations.
  - May require careful tuning of parameters (population size, mutation rate, etc.) for good performance.
  - No guarantee of finding the global optimum.

TSP Representation

Ejercicio 10.4.4. Explain how to define a fitness function for the TSP that evaluates the quality of a route.

A suitable fitness function for the TSP can be defined as the inverse of the total distance of the route. For example, if the total distance of a route is D, the fitness can be calculated as:

$$\text{Fitness} = \frac{1}{D}$$

This way, shorter routes will have higher fitness values, and the genetic algorithm will be incentivized to evolve towards shorter routes.

Genetic Operators for TSP

Ejercicio 10.4.5. Describe suitable crossover and mutation operators for the TSP. Explain why standard crossover and mutation operators might not be effective.

For the TSP, standard crossover and mutation operators (like one-point crossover or bit-flip mutation) are not effective because they can produce invalid routes (e.g., duplicate cities or missing cities). Instead, specialized operators that preserve the permutation structure of the solution are needed. Examples of crossover operators suitable for TSP include:

- Variant of Ordered Crossover (OX): A subrange of the first parent is copied to the child (in the same positions), and the remaining positions are filled with values from the second parent. The variant considers that if a value from the second parent is already in the child, it is skipped until a non-duplicate value is found.

Examples of mutation operators suitable for TSP include:

- Swap Mutation: Two cities in the route are selected and their positions are swapped.
- Inversion Mutation: A segment of the route is selected and reversed.

Ejercicio 10.4.6. Discuss the importance of maintaining valid routes (permutations) during crossover and mutation.

In the TSP, each candidate solution is a permutation of cities. It is crucial to maintain valid routes during crossover and mutation to ensure that each city is visited exactly once. Standard crossover and mutation operators might produce invalid routes (e.g., duplicate cities or missing cities), which would not be meaningful solutions to the TSP. Therefore, specialized operators that preserve the permutation structure are necessary to ensure the integrity of the solutions. Invalid routes would lead to incorrect fitness evaluations and hinder the convergence of the genetic algorithm towards optimal solutions.

Ejercicio 10.4.7. Perform the Maximum Preservative Crossover (MPX) for the two chromosomes `bcabbbbc` and `acabccbb` using the obtained segment: 1 - 4 (0 is the first position), and use the deletion mutation operator.

The Maximum Preservative Crossover (MPX) involves selecting a segment from the first parent and preserving it in the child, while filling the remaining positions with genes from the second parent from the selected segment. For the given chromosomes `bcabbbbc` and `acabccbb`, we select the segment from position 1 to 4 (inclusive) from the first parent, which gives us the segment `cabb`. We then fill the remaining positions with genes from the second parent. The process is shown in Table 10.3.

Parent 1	B	C	A	B	B	B	B	C
Parent 2	A	C	A	B	C	C	B	B
Offspring	C	C	A	B	B	A	B	C

Tabla 10.3: MPX crossover for the given chromosomes and segment 1 - 4.

After performing the MPX crossover, we obtain the offspring `ccabbabc`. Next, we apply the deletion mutation operator, which involves randomly deleting a gene from the offspring. For example, if we randomly select the gene at position 1 (the second "c"), we would delete it, resulting in the mutated offspring `cabbabc`. It should be noted that this mutation may not be valid for some problems, as the resulting chromosome does not have the same length as the original chromosomes.

10.5. K-Means & DBSCAN Clustering

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.5.1. What is the main mathematical difference between K-means and K-medians? Which algorithm is more robust against outliers?

The main mathematical difference between K-means and K-medians lies in the way they calculate the cluster centers. K-means uses the mean (average) of the data points in a cluster to determine the cluster center, while K-medians uses the median (the middle value when the data points are sorted) to determine the cluster center. The K-medians algorithm is more robust against outliers because the median is less affected by extreme values than the mean. In K-means, outliers can significantly skew the cluster centers, while in K-medians, the median will not be influenced as much by outliers, leading to more stable cluster centers.

Ejercicio 10.5.2. Explain the core idea behind DBSCAN. What do the parameters `eps` (ε) and `min_samples` mean?

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that groups together points that are closely packed together, while marking points that lie alone in low-density regions as outliers. The core idea behind DBSCAN is to identify clusters based on the density of data points in the feature space. The two main parameters of DBSCAN are:

- `eps` (ε): This parameter defines the maximum distance between two points for them to be considered as part of the same cluster. It is essentially the radius of the neighborhood around a point.
- `min_samples`: This parameter specifies the minimum number of points required to form a dense region (i.e., a cluster). A point is considered a core point if it has at least `min_samples` points within its ε neighborhood.

Ejercicio 10.5.3. In the context of DBSCAN, define the terms: core point, border point, and noise (outlier).

- Core Point: A core point is a point that has at least `min_samples` points (including itself) within its ε neighborhood. Core points are the central points of clusters.
- Border Point: A border point is a point that is not a core point but is within the ε neighborhood of a core point. Border points are part of a cluster but do not have enough neighboring points to be considered core points themselves.
- Noise (Outlier): A noise point, or outlier, is a point that is neither a core point nor a border point. It does not belong to any cluster and is considered an anomaly in the dataset.

Ejercicio 10.5.4. When does K-Medians (or K-Means) fail at clustering, and when does DBSCAN?

K-Medians (or K-Means) can fail at clustering when the data has non-spherical clusters, varying cluster sizes, or when there are outliers present in the dataset.

These algorithms assume that clusters are spherical and of similar size, which may not always be the case in real-world data. Additionally, K-Means is sensitive to outliers, which can skew the cluster centers and lead to poor clustering results.

10.6. Random Forests

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.6.1. What is the core principle behind Random Forests, and how does it differ from a single decision tree?

The core principle behind Random Forests is the concept of ensemble learning, where multiple decision trees are trained on different subsets of the data and features. Each tree makes a prediction, and the final output is determined by aggregating the predictions (e.g., majority voting for classification or averaging for regression). This approach differs from a single decision tree, which relies on a single model that may be prone to overfitting and may not generalize well to unseen data. Random Forests, by combining multiple trees, can reduce overfitting and improve generalization performance.

Ejercicio 10.6.2. What is Gini impurity?

Gini impurity is a measure of the impurity or disorder in a dataset used in decision tree algorithms. It quantifies how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset. The formula for Gini impurity is:

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

where p_i is the proportion of instances belonging to class i in the subset. A Gini impurity of 0 indicates that all instances belong to a single class, while a Gini impurity close to 1 indicates that the classes are evenly distributed.

Ejercicio 10.6.3. Explain the concepts of "bagging" and "feature randomness" in the context of Random Forests.

"Bagging" (Bootstrap Aggregating) is a technique used in Random Forests where multiple subsets of the training data are created by sampling with replacement. Each decision tree in the forest is trained on a different subset of the data, which helps to reduce variance and improve model robustness.

"Feature randomness" refers to the process of selecting a random subset of features for each split in the decision trees. This means that when a tree is being built, it does not consider all features for making splits but rather a random selection of them. This introduces additional diversity among the trees, which can further reduce overfitting and improve the overall performance of the Random Forest.

Ejercicio 10.6.4. How do Random Forests handle classification and regression tasks differently?

Random Forests handle classification and regression tasks by using different aggregation methods for the final predictions. In classification tasks, Random Forests use majority voting, where the class that receives the most votes from the individual trees is selected as the final prediction. In regression tasks, Random Forests use averaging, where the predicted values from all the trees are averaged to produce the final output. This allows Random Forests to effectively handle both types of tasks while maintaining their robustness and accuracy.

Ejercicio 10.6.5. What are some key hyperparameters of Random Forests, and how do they impact model performance?

Some key hyperparameters of Random Forests include:

- The number of trees in the forest. Increasing the number of trees can improve performance but also increases computational cost.
- The maximum depth of the trees. Limiting the depth can help prevent overfitting, while allowing deeper trees can capture more complex patterns in the data.
- The minimum number of samples required to split a node. Higher values can lead to simpler trees and reduce overfitting, while lower values can create more complex trees that may overfit the training data.
- The number of features to consider when looking for the best split. This can impact the diversity of the trees and, consequently, the overall performance of the model.

Ejercicio 10.6.6. Discuss the advantages and disadvantages of using Random Forests compared to other machine learning algorithms.

Advantages of Random Forests include:

- They are robust to overfitting due to the ensemble nature of the model.
- They can handle both classification and regression tasks effectively.
- They can capture complex relationships in the data without requiring extensive feature engineering.
- They provide feature importance measures, which can help identify the most relevant features in the dataset.

Disadvantages of Random Forests include:

- They can be computationally expensive, especially with a large number of trees and features.
- They may not perform well on datasets with a large number of irrelevant features, as the random selection of features can lead to less informative splits.
- They can be less interpretable than simpler models like decision trees or linear regression, making it harder to understand the underlying relationships in the data.

10.7. Regression

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.7.1. What is the primary goal of linear regression?

Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables. The primary goal of linear regression is to find the best-fitting line (or hyperplane in higher dimensions) that describes the relationship between the independent variables and the dependent variable. This line can then be used to make predictions about the dependent variable based on new values of the independent variables.

Ejercicio 10.7.2. Explain the difference between independent and dependent variables in a regression model.

In a regression model, the independent variable(s) (also known as predictor(s) or feature(s)) are the variable(s) that are used to predict the value of the dependent variable. The dependent variable (also known as the response variable or target variable) is the variable that we are trying to predict or explain. The independent variables are assumed to have an influence on the dependent variable, and the goal of regression is to quantify this relationship.

Ejercicio 10.7.3. What are the assumptions of linear regression, and why are they important?

The assumptions of linear regression are:

- Linearity: The relationship between the independent and dependent variables is linear.
- Independence: The residuals (errors) are independent of each other.
- No multicollinearity: The independent variables are not highly correlated with each other.
- Normality: The residuals are normally distributed.
- Homoscedasticity: The variance of the residuals is constant across all levels of the independent variables.

These assumptions are important because they ensure that the estimates of the regression coefficients are unbiased and efficient. Violations of these assumptions can lead to incorrect conclusions and predictions from the regression model.

Ejercicio 10.7.4. How do Mean Squared Error (MSE) and R-squared (R2) measure the performance of a regression model?

Mean Squared Error (MSE) is a measure of the average squared difference between the observed actual outcomes and the predicted values from the regression model. A lower MSE indicates a better fit of the model to the data.

R-squared (R2) is a statistical measure that represents the proportion of the variance in the dependent variable that is explained by the independent variables in the model. An R2 value of 1 indicates that the model perfectly explains the variance

in the dependent variable, while an R2 value of 0 indicates that the model does not explain any of the variance. Higher R2 values indicate a better fit of the model to the data.

Ejercicio 10.7.5. What is the purpose of regularization (L1 and L2) in regression, and how do they differ?

Regularization is a technique used to prevent overfitting in regression models by adding a penalty term to the loss function. The two most common types of regularization are L1 (Lasso) and L2 (Ridge) regularization.

- L1 regularization (Lasso) adds a penalty equal to the absolute value of the coefficients, which can lead to some coefficients being exactly zero. This means that L1 regularization can perform feature selection by effectively removing some features from the model.
- L2 regularization (Ridge) adds a penalty equal to the square of the coefficients, which tends to shrink the coefficients towards zero but does not set them exactly to zero. This means that L2 regularization can help reduce the impact of less important features without completely removing them from the model, reducing the impact of multicollinearity and improving the stability of the model.

In summary, both L1 and L2 regularization aim to improve the generalization of the regression model by preventing overfitting, but they do so in different ways: L1 can lead to sparse models with feature selection, while L2 shrinks coefficients without eliminating features.

10.8. Neural Network

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.8.1. Explain the role of activation functions in a neural network. Describe two common activation functions (e.g., ReLU, sigmoid, tanh) and discuss a scenario where one might be preferred over the other.

Activation functions introduce non-linearity into the neural network, allowing it to learn complex patterns in the data. Without activation functions, a neural network would essentially be a linear model, regardless of the number of layers. Two common activation functions are:

- ReLU (Rectified Linear Unit): Defined as $f(x) = \text{máx}(0, x)$, ReLU is computationally efficient and helps mitigate the vanishing gradient problem. It is often preferred in hidden layers of deep networks.
- Sigmoid: Defined as $f(x) = \frac{1}{1+e^{-x}}$, the sigmoid function maps input values to a range between 0 and 1. It is commonly used in the output layer for binary classification problems, as it can be interpreted as a probability.

Ejercicio 10.8.2. Describe the process of backpropagation during the training of a neural network.

Backpropagation is a supervised learning algorithm used to train neural networks. It consists of several steps:

1. Forward Pass: The input data is passed through the network to compute the output.
2. Loss Calculation: The output is compared to the true labels using a loss function (e.g., Mean Absolute Error, Cross-Entropy Loss) to calculate the error.
3. Backward Pass: The error is propagated backward through the network to compute the gradients of the loss with respect to each weight in the network.
4. Weight Update: The weights are updated using an optimization algorithm (e.g., Gradient Descent) based on the computed gradients to minimize the loss.

Ejercicio 10.8.3. What is the purpose of a loss function in neural network training?

A loss function is a mathematical function that quantifies the difference between the predicted output of the neural network and the true labels. The purpose of the loss function is to provide a measure of how well the model is performing, guiding the training process by indicating how much the model's predictions deviate from the actual values. During training, the optimization algorithm seeks to minimize the loss function, which in turn improves the model's performance on the training data.

Ejercicio 10.8.4. Explain the difference between a classification task and a regression task.

A classification task involves predicting a discrete label or category for each input instance. For example, classifying images of animals into categories such as "cat",

"dog", or "bird". The output of a classification model is typically a probability distribution over the possible classes.

A regression task, on the other hand, involves predicting a continuous value for each input instance. For example, predicting the price of a house based on its features (size, location, etc.). The output of a regression model is a single continuous value or a vector of continuous values.

Ejercicio 10.8.5. Discuss the concepts of overfitting and underfitting in the context of neural network training. How can you identify these issues by analyzing the training and validation loss curves? What are two common techniques to mitigate overfitting?

- Overfitting: Occurs when a model learns the training data too well, including its noise and outliers, resulting in poor generalization to new, unseen data. It can be identified by a low training loss and a high validation loss.
- Underfitting: Occurs when a model is too simple to capture the underlying patterns in the data, resulting in poor performance on both the training and validation sets. It can be identified by high training and validation losses.

Two common techniques to mitigate overfitting are:

- Regularization: Techniques such as L1 or L2 regularization add a penalty term to the loss function to discourage complex models and prevent overfitting.
- Dropout: A technique that randomly drops a fraction of the neurons during training, which helps prevent the model from becoming too reliant on any particular set of features and encourages it to learn more robust representations.

Ejercicio 10.8.6. Why does a neural network need an activation function in the first place? What would happen if we completely omitted it?

A neural network needs an activation function to introduce non-linearity into the model. Without activation functions, the network would only be able to learn linear relationships between the input and output, regardless of the number of layers. This would severely limit the model's capacity to capture complex patterns in the data. If we completely omitted activation functions, the network would effectively become a linear regression model, and it would not be able to solve problems that require learning non-linear decision boundaries, such as image classification or speech recognition.

Ejercicio 10.8.7. What is Softmax? (Briefly explain its purpose and mathematical functionality).

Softmax is an activation function that converts a vector of raw scores (logits) into a probability distribution over multiple classes. It is commonly used in the output layer of multi-class classification neural networks. The Softmax function takes as input a vector of real numbers and outputs a vector of values between 0 and 1 that sum to 1, representing the probabilities of each class. It includes a parameter T (temperature) that can be used to control the ""sharpness" of the output distribution. The mathematical formula for Softmax is given by:

$$\text{Softmax}(x_i) = \frac{e^{x_i/T}}{\sum_j e^{x_j/T}}$$

where j indexes over all classes, x_i is the input score for class i, and T is the temperature parameter. A lower temperature results in a more confident (peaked) distribution, while a higher temperature produces a softer (more uniform) distribution.

Ejercicio 10.8.8. What exactly does the "Loss" measure during training, and why does the optimizer try to make this value as small as possible?

The "Loss" measures the discrepancy between the predicted output of the neural network and the true labels of the training data. It quantifies how well the model is performing on the training examples. A lower loss indicates that the model's predictions are closer to the actual values, while a higher loss indicates greater error in the predictions. The optimizer tries to minimize the loss value because doing so improves the model's performance. By reducing the loss, the model learns to make more accurate predictions, which is the ultimate goal of training a neural network. A smaller loss value generally corresponds to better generalization to unseen data, leading to improved performance on real-world tasks.

Ejercicio 10.8.9. Provide the mathematical formula for the Cross-Entropy loss function (for a single sample in multi-class classification) and explain its individual components.

The Cross-Entropy loss function for a single sample in multi-class classification is given by the formula:

$$L = -\sum_{i=1}^{C} y_i \log(p_i)$$

where:

- C is the number of classes.
- y_i is the true label for class i, which is 1 if the sample belongs to class i and 0 otherwise (one-hot encoded).
- p_i is the predicted probability for class i, obtained from the Softmax output of the neural network.

Ejercicio 10.8.10 (Bonus)**.** Explain the concept of a batch size in the context of training a neural network with a DataLoader. What are the potential benefits and drawbacks of using a smaller versus a larger batch size?

The batch size is the number of training examples used in one forward and backward pass of the training process before the network's weights are updated. A `DataLoader` divides the entire training dataset into smaller batches.

Some benefits of using a smaller batch size include:

- Given that the estimation of the gradient is noisier, it can help the model escape local minima and improve generalization.
- Smaller batches require less memory on the GPU or CPU.
- The model's weights are updated more frequently, which can lead to faster learning in some cases.

Ejercicio 10.8.11 (Bonus)**.** What is regularization in the context of neural networks? Describe one common regularization technique (e.g., dropout) and explain how it can help improve the generalization ability of a model.

Regularization refers to techniques used to prevent overfitting in machine learning models, including neural networks. They add extra constraints or information to the learning algorithm to prevent the model from becoming too complex and fitting the training data too closely. The goal is to improve the model's ability to generalize to unseen data.

An example of a common regularization technique is *dropout*. Dropout is a regularization technique where, during each training iteration, a randomly selected proportion of neurons in a layer are set to zero. This means that these neurons are not activated in the forward pass and do not participate in the backward pass.

10.9. Gated Recurrent Unit (GRU)

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.9.1. Describe the main components of a GRU cell and their function.

A GRU cell consists of three main components:

- Update Gate: This gate determines how much of the previous hidden state should be retained and how much of the new candidate hidden state should be used. It helps the model to decide whether to keep the old information or to update it with new information.
- Reset Gate: This gate controls how much of the previous hidden state should be ignored when calculating the new candidate hidden state. It allows the model to reset its memory when necessary, which can help in capturing short-term dependencies.
- Candidate Hidden State: This is a potential new hidden state that is computed based on the current input and the previous hidden state, modulated by the reset gate.

The final hidden state is then computed as a combination of the previous hidden state and the candidate hidden state, weighted by the update gate.

Ejercicio 10.9.2. How do the update and reset gates in a GRU cell differ in their operation?

The update gate in a GRU cell controls the extent to which the previous hidden state is retained versus how much of the new candidate hidden state is used. It essentially decides whether to keep the old information or to update it with new information. In contrast, the reset gate determines how much of the previous hidden state should be ignored when calculating the new candidate hidden state. It allows the model to reset its memory when necessary, which can help in capturing short-term dependencies. In summary, the update gate manages the flow of information from the past to the future, while the reset gate manages how much of the past information is considered when generating new candidate states.

Ejercicio 10.9.3. Name two advantages of GRUs over simple RNNs.

Two advantages of GRUs over simple RNNs (which are really related) are:

- Mitigation of the Vanishing Gradient Problem: GRUs, like LSTMs, use gating mechanisms that help to preserve gradients during backpropagation.
- Learning Long-Term Dependencies: GRUs can capture long-term dependencies in sequential data more effectively than simple RNNs, which struggle with this due to the vanishing gradient problem.

Ejercicio 10.9.4. Compare GRUs and LSTMs with regard to their structure and complexity.

GRUs and LSTMs are both types of recurrent neural network architectures designed to address the vanishing gradient problem. However, they differ in their structure and complexity:

- Structure: LSTMs have three gates (input, forget, and output gates) and a cell state, while GRUs have only two gates (update and reset gates). This makes GRUs simpler in terms of architecture compared to LSTMs.
- Complexity: Due to their simpler structure, GRUs typically have fewer parameters than LSTMs, which can lead to faster training and inference times. However, the choice between GRUs and LSTMs often depends on the specific task and dataset, as LSTMs may perform better on certain tasks that require modeling more complex dependencies.

Ejercicio 10.9.5. Give a typical application example for GRUs.

A typical application example for GRUs is in natural language processing (NLP) tasks, such as:

- Machine Translation: GRUs can be used in sequence-to-sequence models for translating text from one language to another, where they help to capture the dependencies between words in a sentence.
- Speech Recognition: GRUs can be employed in models that convert spoken language into text, as they can effectively model the temporal dependencies in audio data.
- Text Generation: GRUs can be used in language models to generate coherent and contextually relevant text based on a given input prompt.

Ejercicio 10.9.6. How do GRUs help to reduce the vanishing gradient problem?

GRUs help to reduce the vanishing gradient problem through their gating mechanisms, specifically the update and reset gates. The update gate allows the model to retain information from previous time steps, which helps to preserve gradients during backpropagation. The reset gate allows the model to ignore irrelevant past information when generating new candidate hidden states, which can also help to maintain a stronger gradient signal. By controlling the flow of information and allowing for long-term dependencies, GRUs mitigate the vanishing gradient problem that simple RNNs often face when trying to learn from long sequences.

10.10. Convolutional Neural Networks (CNNs)

The Python part of this exercise is available in this Jupyter Notebook.

Ejercicio 10.10.1 (Convolutional Layer)**.** What is the main task of a convolutional layer and how does it help the network find patterns (like edges or shapes) in an image?

The main task of a convolutional layer is to extract features from the input data (such as images) by applying convolution operations using filters (kernels). These filters slide over the input image and perform element-wise multiplications and summations, allowing the network to detect local patterns such as edges, textures, and shapes. By learning different filters during training, the convolutional layer can capture various features at different levels of abstraction, which helps the network understand and recognize complex patterns in the data.

Ejercicio 10.10.2 (Filter)**.** What does a filter (kernel) do in a convolutional layer as it moves step-by-step across the image?

A filter (kernel) in a convolutional layer is a small matrix of weights that is applied to the input image through a process called convolution. As the filter moves step-by-step (or slides) across the image, it performs element-wise multiplications between the filter values and the corresponding pixel values of the image, followed by summing these products to produce a single output value. This operation allows the filter to detect specific features in different regions of the image, such as edges, corners, or textures, depending on the learned weights of the filter.

Ejercicio 10.10.3 (Image Channels)**.** What is the difference between a grayscale image and a color image (RGB) regarding the number of channels the network has to process?

A grayscale image has a single channel, where each pixel represents the intensity of light (brightness) at that point, typically ranging from black to white. In contrast, a color image (RGB) has three channels corresponding to the red, green, and blue components of each pixel. Each channel contains intensity values for its respective color, allowing the network to process and learn from the combined information of all three channels to capture color features in addition to spatial patterns.

Ejercicio 10.10.4 (Stride)**.** What does the term stride mean and how does the output size change when you increase the stride?

The term stride refers to the number of pixels by which the filter moves (or slides) across the input image during the convolution operation. A stride of 1 means the filter moves one pixel at a time, while a stride of 2 means it moves two pixels at a time. Increasing the stride reduces the spatial dimensions of the output feature map because fewer positions are sampled from the input image. As a result, larger strides lead to smaller output sizes, which can help reduce computational complexity but may also result in loss of spatial information.

Ejercicio 10.10.5 (Padding)**.** Why do we often add an artificial border (e.g., made of zeros) around the image before applying convolution?

We often add an artificial border (e.g., made of zeros) around the image given two main reasons:

- To preserve the spatial dimensions of the output feature map. Without padding, the size of the feature map decreases after each convolution operation, which can lead to a loss of information, especially in deeper networks.

- To ensure that the filter can be applied to the edges of the image. Without padding, the filter has fewer neighboring pixels to convolve with at the borders, which can lead to artifacts in the feature map and may not capture important edge features.

Padding helps maintain the integrity of the input data and allows the network to learn features from the entire image, including the edges.

Ejercicio 10.10.6 (Max Pooling)**.** How does max pooling work and why does it help reduce the amount of data?

Max pooling is a downsampling operation that reduces the spatial dimensions of the feature maps while retaining the most important information. It works by dividing the input feature map into non-overlapping regions (usually squares) and selecting the maximum value from each region to create a smaller output feature map. This process helps reduce the amount of data by summarizing the most prominent features in each region, which can lead to lower computational complexity, reduced memory usage, and improved robustness to variations in the input data, such as translations and distortions.

Ejercicio 10.10.7 (Purpose of Pooling)**.** Why does a pooling layer almost always follow directly after a convolutional layer in a CNN architecture?

A pooling layer almost always follows directly after a convolutional layer in a CNN architecture because it serves to downsample the feature maps produced by the convolutional layer. This downsampling reduces the spatial dimensions of the feature maps, which helps to decrease computational complexity and memory usage. Additionally, pooling layers help to make the network more robust to variations in the input data, such as translations and distortions, by retaining the most important features while discarding less relevant information. By following a convolutional layer with a pooling layer, the network can effectively capture hierarchical features at different levels of abstraction.

Observación. It is true that a bigger stride in the convolutional layer can also reduce the size of the feature maps, but pooling layers provide additional benefits. Pooling layers help to retain the most important features while discarding less relevant information, which can improve the network's ability to generalize to new data.

Ejercicio 10.10.8 (Flattening)**.** What happens during flattening and why must the data be made flat before it flows into the final layers?

During flattening, the multi-dimensional feature maps produced by the convolutional and pooling layers are transformed into a one-dimensional vector. This process is necessary because the final layers of a CNN, typically fully connected (dense) layers, expect input in a flat format. Flattening allows the network to combine the learned features from the previous layers and pass them to the fully connected layers for further processing, such as classification or regression. By converting the

spatially structured data into a flat vector, the network can effectively learn complex relationships between the features and make predictions based on the extracted information.

Ejercicio 10.10.9 (Fully Connected Layer)**.** What is the role of the classic, fully connected layers (dense layers) at the very end of a CNN?

The role of the classic, fully connected layers (dense layers) at the very end of a CNN is to perform high-level reasoning and make final predictions based on the features extracted by the preceding convolutional and pooling layers. These layers take the flattened feature vector as input and learn complex relationships between the features through weighted connections. The fully connected layers combine the learned features to produce output values, such as class probabilities in classification tasks or continuous values in regression tasks. They serve as the decision-making part of the network, translating the learned representations into actionable outputs.

Ejercicio 10.10.10 (Number of Filters)**.** Why do we usually use more filters in the deeper layers of a CNN compared to the very first layers?

We usually use more filters in the deeper layers of a CNN compared to the very first layers because deeper layers are responsible for capturing more complex and abstract features of the input data. The initial layers typically learn simple features such as edges and textures, which can be represented with fewer filters. As we move deeper into the network, the features become more intricate, requiring a larger number of filters to capture various combinations of lower-level features and to represent higher-level concepts. By increasing the number of filters in deeper layers, the network can learn a richer set of features, improving its ability to recognize complex patterns and make accurate predictions.